



UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación

Ingeniería en Software

PRÁCTICA EXPERIMENTAL IV

**Gestión de información en aplicaciones web mediante
objetos del ambiente y acceso a datos**

Materia: Aplicaciones Web

Docente: Gleiston Guerrero Ulloa

Período Académico: SPA 2025-2026

Integrantes:

- Castro Coello Mario
- Mendoza Moreira Andy
- Oña Zapata Ronald
- Reinoso Vélez Eduardo

Quevedo - Ecuador
Enero 2026

Investigación bibliográfica: Gestión de la información en aplicaciones web

La gestión de información en aplicaciones web representa uno de los pilares fundamentales en el desarrollo de software moderno, constituyendo la base sobre la cual se construyen sistemas robustos, escalables y seguros. En el contexto actual, donde las aplicaciones web se han convertido en la principal forma de interacción entre usuarios y sistemas de información [1], comprender los mecanismos de almacenamiento, procesamiento y transmisión de datos resulta esencial para desarrollar soluciones empresariales de calidad. Este documento presenta una investigación bibliográfica sobre los conceptos, tecnologías y mejores prácticas relacionadas con la gestión de información en aplicaciones web, enfocándose particularmente en el ecosistema Java y Spring Boot como framework de desarrollo principal. La investigación aborda desde los fundamentos teóricos de las arquitecturas web hasta aspectos avanzados como procedimientos almacenados y buenas prácticas de seguridad [2], [3]. El objetivo principal de esta investigación es proporcionar un marco teórico comprehensivo que sustente la práctica experimental sobre gestión de información mediante objetos del ambiente y acceso a datos, permitiendo a los estudiantes comprender cómo se almacena, comparte y gestiona la información entre diferentes usuarios y roles de usuario, aplicando tecnologías de desarrollo web contemporáneas según los estándares de la industria.

1. Aplicaciones web y gestión de información

Las aplicaciones web modernas constituyen sistemas complejos que requieren una gestión sofisticada de la información para mantener su funcionalidad y rendimiento. La arquitectura basada en microservicios, ampliamente adoptada en sistemas modernos donde la escalabilidad y el mantenimiento del código son prioridades, ha transformado la manera en que se diseñan y despliegan las aplicaciones empresariales [4], [5]. Spring Boot ha emergido como uno de los frameworks más prominentes para el desarrollo de estas aplicaciones, simplificando significativamente la creación de sistemas backend mediante configuraciones automáticas y convenciones predefinidas [2], [6]. Este framework reduce el esfuerzo de codificación redundante y permite a los desarrolladores centrarse en la lógica de negocio en lugar de en aspectos técnicos de infraestructura [5], [7].

1.1. Concepto de aplicación web

Una aplicación web se define como un sistema de software que se ejecuta en un servidor y es accesible a través de navegadores web mediante redes de comunicación, típicamente Internet. A diferencia de las aplicaciones de escritorio tradicionales, las aplicaciones web no requieren instalación en el dispositivo del usuario y pueden ser accedidas desde cualquier ubicación con conectividad de red [1]. El patrón Modelo-Vista-Controlador (MVC) divide una aplicación en tres componentes interconectados, donde el modelo gestiona los datos y la lógica de negocio, la vista presenta la información al usuario, y el controlador maneja las interacciones del usuario [8]. Este patrón arquitectónico se ha consolidado como estándar en el desarrollo web debido a su capacidad para separar responsabilidades, facilitando el desarrollo paralelo y el mantenimiento del código [1], [9].

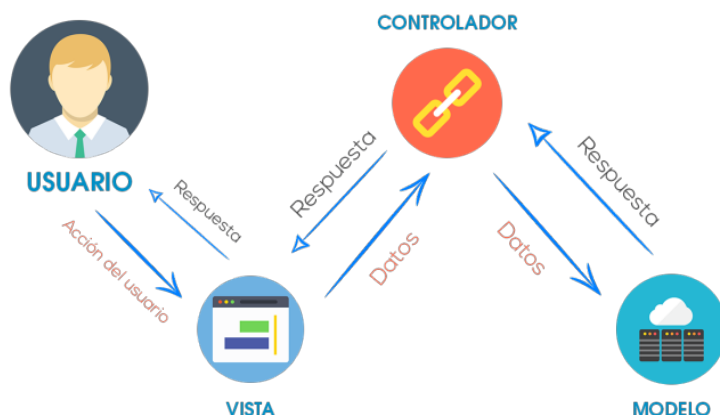


Figura 1: Arquitectura del patrón Modelo-Vista-Controlador en aplicaciones web

1.2. Importancia de la gestión de información

La gestión efectiva de información constituye un componente crítico para el éxito de cualquier aplicación web empresarial, impactando directamente en el rendimiento, seguridad y escalabilidad del sistema. La implementación de patrones adecuados de gestión de información ayuda a evitar errores comunes relacionados con el estado de la aplicación y el almacenamiento de datos, mejorando la confiabilidad y mantenibilidad del sistema [2], [6]. Los frameworks de persistencia como Spring Data simplifican radicalmente el acceso a datos en aplicaciones Java, proporcionando abstracciones que permiten a los desarrolladores trabajar con objetos en lugar de consultas SQL directas [10], [11]. Una gestión adecuada de la información permite además optimizar el rendimiento mediante estrategias de caché, planes de consulta eficientes y minimización de transferencias de datos entre la aplicación y la base de datos [11], [12], [13].

1.3. Entornos multiusuario

Los entornos multiusuario representan escenarios donde múltiples usuarios interactúan simultáneamente con una aplicación web, cada uno manteniendo su propia sesión y contexto de trabajo independiente. La gestión de sesiones es fundamental para mantener la seguridad, rendimiento y escalabilidad de aplicaciones web, ya que permite rastrear las interacciones de los usuarios y preservar su estado a través de múltiples solicitudes HTTP stateless [14]. Las aplicaciones web deben implementar mecanismos robustos de gestión de sesiones para prevenir ataques de secuestro de sesión, fijación de sesión y otros vectores de amenaza comunes [15]. En arquitecturas multiusuario, resulta crucial establecer políticas claras de expiración de sesiones, regeneración de identificadores tras la autenticación y uso de comunicaciones cifradas mediante HTTPS para proteger los tokens de sesión en tránsito [16], [17].

2. Arquitectura de aplicaciones web

La arquitectura de aplicaciones web define la estructura organizacional y los patrones de interacción entre los componentes que conforman un sistema, estableciendo cómo se distribuyen las responsabilidades y se gestiona el flujo de información. Spring Boot

promueve arquitecturas bien estructuradas mediante la separación clara de responsabilidades en capas, facilitando la escalabilidad horizontal, el mantenimiento del código y la implementación de pruebas automatizadas [2], [6]. Una arquitectura bien diseñada permite que los equipos de desarrollo trabajen de manera eficiente en diferentes módulos del sistema sin generar conflictos, facilita la evolución del software y simplifica la integración de nuevas funcionalidades [6], [9].

2.1. Modelo cliente-servidor

El modelo cliente-servidor constituye un paradigma arquitectónico fundamental donde las tareas se distribuyen entre proveedores de recursos o servicios (servidores) y solicitantes de servicios (clientes). En este modelo, el cliente inicia las solicitudes enviando peticiones HTTP al servidor, el cual procesa la solicitud, ejecuta la lógica de negocio necesaria y devuelve una respuesta apropiada [1], [8]. La arquitectura cliente-servidor permite la centralización de recursos y datos en el servidor, facilitando el control de acceso, la consistencia de la información y la implementación de medidas de seguridad [4], [7]. Este patrón ha evolucionado significativamente con la adopción de arquitecturas RESTful que utilizan los métodos HTTP (GET, POST, PUT, DELETE) para implementar operaciones CRUD sobre recursos del servidor [18], [19].

2.2. Capas de una aplicación web

La arquitectura en capas constituye uno de los patrones más comunes en el desarrollo de software, dividiendo la aplicación en niveles con responsabilidades específicas y bien definidas. La arquitectura de tres capas organiza el código en una capa de presentación que maneja la interfaz de usuario, una capa de lógica de negocio que implementa las reglas del dominio, y una capa de acceso a datos que gestiona la persistencia [8], [9]. En aplicaciones Spring Boot, esta separación se manifiesta típicamente en controladores (capa de presentación), servicios (lógica de negocio) y repositorios (acceso a datos), promoviendo el bajo acoplamiento y la alta cohesión [2], [6]. El patrón MVC se integra con la arquitectura en capas, donde el controlador maneja la lógica de presentación, el modelo representa tanto la capa de negocio como la de datos, y la vista gestiona la representación visual [1], [8].

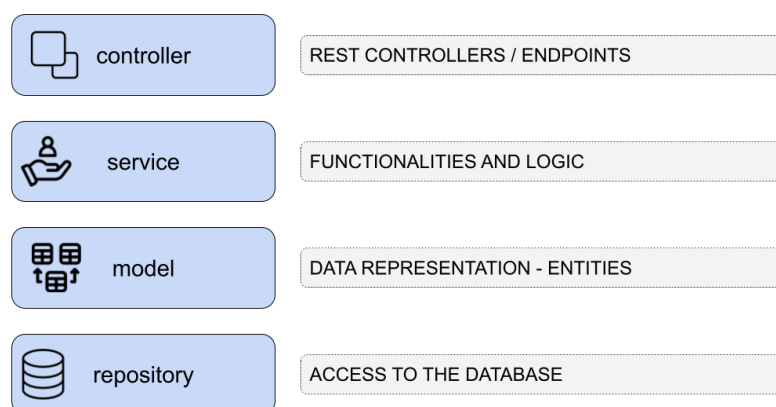


Figura 2: Arquitectura en capas de una aplicación Spring Boot

2.3. Flujo de información cliente-servidor

El flujo de información en aplicaciones web sigue un ciclo bien definido que comienza cuando el cliente envía una solicitud HTTP al servidor. Al recibir la solicitud, el controlador en el servidor la procesa, extrae los parámetros necesarios y delega la ejecución de la lógica de negocio a la capa de servicios [18], [20]. La capa de servicios interactúa con la capa de persistencia para recuperar o modificar datos en la base de datos, aplicando transformaciones y validaciones según las reglas del negocio [2], [6]. Finalmente, el servidor construye una respuesta que puede incluir datos en formato JSON o XML, códigos de estado HTTP y encabezados, enviándola de vuelta al cliente para su procesamiento y presentación al usuario [18], [19].

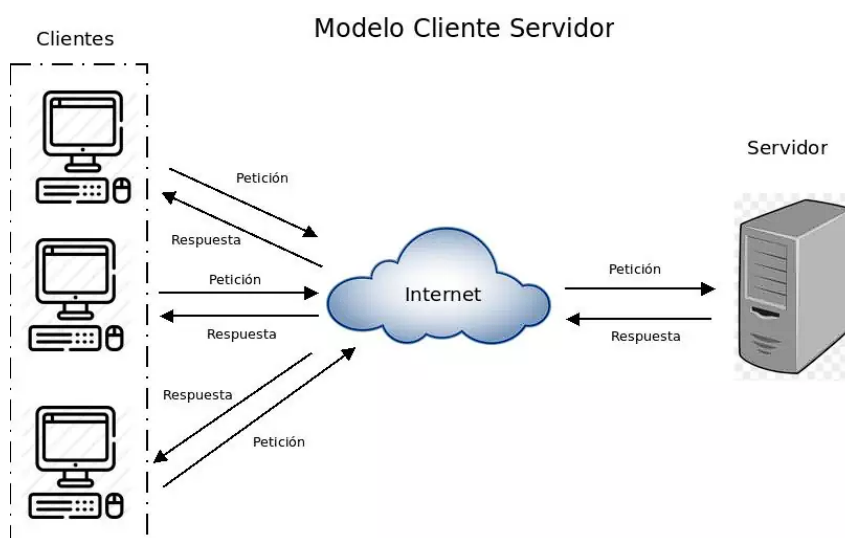


Figura 3: Flujo de información en el modelo cliente-servidor de aplicaciones web

3. Objetos del ambiente web

Los objetos del ambiente web son componentes fundamentales que permiten gestionar el ciclo de vida y el alcance de la información en aplicaciones web, proporcionando mecanismos para almacenar y compartir datos en diferentes contextos de ejecución. Estos objetos definen el ámbito de persistencia de los datos, desde información específica de una petición individual hasta datos compartidos entre todos los usuarios de la aplicación [18], [20]. La correcta utilización de estos objetos es esencial para implementar funcionalidades como autenticación de usuarios, mantenimiento de estado de sesión y comunicación eficiente entre las capas de la aplicación [17], [21].

3.1. Objeto *Request*

El objeto *Request* encapsula toda la información asociada con una solicitud HTTP específica enviada por el cliente al servidor. Este objeto contiene los parámetros de consulta, encabezados HTTP, método de solicitud (GET, POST, PUT, DELETE), cookies, y el cuerpo de la solicitud cuando aplica [18], [19]. En Spring Boot, el objeto *Request* se gestiona mediante anotaciones como `@RequestParam` para extraer parámetros de consulta, `@RequestBody` para deserializar el cuerpo de la solicitud en objetos Java,

y `@RequestHeader` para acceder a encabezados HTTP específicos [2], [6]. El alcance del objeto `Request` se limita al procesamiento de una única solicitud, destruyéndose automáticamente una vez que se envía la respuesta al cliente.

```

1 @RestController
2 @RequestMapping("/api/users")
3 public class UserController {
4
5     @GetMapping("/search")
6     public ResponseEntity<List<User>> searchUsers(
7         @RequestParam String name,
8         @RequestParam(required = false) Integer age,
9         @RequestHeader("User-Agent") String userAgent) {
10
11         // Acceso a parametros del request
12         System.out.println("Buscando: " + name);
13         System.out.println("Edad: " + age);
14         System.out.println("Navegador: " + userAgent);
15
16         List<User> users = userService.findByName(name, age);
17         return ResponseEntity.ok(users);
18     }
19
20     @PostMapping
21     public ResponseEntity<User> createUser(
22         @RequestBody User user) {
23         // Deserializacion automatica del JSON del request
24         User savedUser = userService.save(user);
25         return ResponseEntity.status(HttpStatus.CREATED)
26             .body(savedUser);
27     }
28 }

```

Listing 1: Ejemplo de uso del objeto `Request` en Spring Boot

3.2. Objeto *Response*

El objeto `Response` representa la respuesta que el servidor envía de vuelta al cliente después de procesar una solicitud. Este objeto permite configurar el código de estado HTTP (200 para éxito, 404 para no encontrado, 500 para errores del servidor), establecer encabezados de respuesta, definir cookies, y escribir el cuerpo de la respuesta que puede contener HTML, JSON, XML u otros formatos [18], [19]. Spring Boot facilita la construcción de respuestas mediante anotaciones como `@ResponseBody` que serializa automáticamente objetos Java a JSON, y `ResponseEntity` que proporciona control completo sobre todos los aspectos de la respuesta HTTP [2], [6]. El manejo adecuado del objeto `Response` es crucial para implementar APIs RESTful que comuniquen claramente el resultado de las operaciones al cliente.

```

1 @RestController
2 @RequestMapping("/api/products")
3 public class ProductController {
4
5     @Autowired
6     private ProductService productService;
7
8     @GetMapping("/{id}")
9     public ResponseEntity<Product> getProduct(
10         @PathVariable Long id) {
11
12         Optional<Product> product = productService.findById(id);
13
14         if (product.isPresent()) {
15             // Response exitoso con codigo 200
16             return ResponseEntity.ok()
17                 .header("X-Custom-Header", "ProductFound")
18                 .body(product.get());
19         } else {
20             // Response con codigo 404
21             return ResponseEntity.notFound()
22                 .header("X-Error-Message", "Product not found")
23                 .build();
24         }
25     }
26
27     @DeleteMapping("/{id}")
28     public ResponseEntity<Void> deleteProduct(
29         @PathVariable Long id) {
30
31         productService.delete(id);
32         // Response con codigo 204 (No Content)
33         return ResponseEntity.noContent().build();
34     }
35 }

```

Listing 2: Ejemplo de uso del objeto Response en Spring Boot

3.3. Objeto *Session*

El objeto Session permite mantener información específica del usuario a través de múltiples solicitudes HTTP, superando la naturaleza stateless del protocolo HTTP. Las sesiones almacenan datos como credenciales de autenticación, preferencias del usuario, carrito de compras y cualquier información que necesite persistir durante la interacción del usuario con la aplicación [17], [22]. En Spring Boot, la gestión de sesiones se implementa mediante la anotación `@SessionAttributes` que especifica qué atributos del modelo deben almacenarse en la sesión HTTP [2], [6]. Las sesiones tienen un tiempo de vida configurable y se destruyen cuando expiran por inactividad, cuando el usuario cierra el navegador, o cuando se invalidan explícitamente mediante logout [17], [21].

```

1 @Controller
2 @SessionAttributes({"currentUser", "shoppingCart"})
3 public class ShoppingController {
4
5     @GetMapping("/login")
6     public String login(@RequestParam String username,
7                         Model model,
8                         HttpSession session) {
9
10         User user = userService.authenticate(username);
11
12         // Almacenar en sesion
13         model.addAttribute("currentUser", user);
14         session.setAttribute("loginTime",
15                             LocalDateTime.now());
16
17         // Configurar timeout de sesion (30 minutos)
18         session.setMaxInactiveInterval(1800);
19
20         return "redirect:/dashboard";
21     }
22
23     @PostMapping("/cart/add")
24     public String addToCart(@RequestParam Long productId,
25                             @ModelAttribute("shoppingCart")
26                             ShoppingCart cart) {
27
28         // El carrito persiste en la sesion
29         cart.addProduct(productId);
30         return "redirect:/cart";
31     }
32
33     @GetMapping("/logout")
34     public String logout(HttpSession session) {
35         // Invalidar sesion
36         session.invalidate();
37         return "redirect:/login";
38     }
39 }

```

Listing 3: Ejemplo de uso del objeto Session en Spring Boot

3.4. Objeto *Application*

El objeto *Application*, también conocido como *ServletContext*, representa un ámbito de almacenamiento compartido entre todos los usuarios y sesiones de la aplicación web. Este objeto persiste mientras la aplicación web esté en ejecución y se utiliza para almacenar configuraciones globales, contadores de visitantes, conexiones a bases de datos compartidas y cualquier recurso que deba ser accesible para todos los componentes de la aplicación [9], [20]. Los datos almacenados en el ámbito de *Application* son visibles para

todas las solicitudes y sesiones, lo que requiere consideraciones especiales de sincronización cuando múltiples hilos acceden simultáneamente a estos datos compartidos [6].

```

1 @Component
2 public class ApplicationScopeManager {
3
4     @Autowired
5     private ServletContext servletContext;
6
7     public void initializeAppConfig() {
8         // Datos compartidos a nivel de aplicacion
9         servletContext.setAttribute("appVersion", "2.0.1");
10        servletContext.setAttribute("visitorCount",
11                                    new AtomicLong(0));
12        servletContext.setAttribute("startupTime",
13                                    LocalDateTime.now());
14    }
15
16    public void incrementVisitorCount() {
17        AtomicLong counter = (AtomicLong)
18            servletContext.getAttribute("visitorCount");
19        counter.incrementAndGet();
20    }
21
22    public Long getVisitorCount() {
23        AtomicLong counter = (AtomicLong)
24            servletContext.getAttribute("visitorCount");
25        return counter.get();
26    }
27 }

```

Listing 4: Ejemplo de uso del objeto Application en Spring Boot

4. Manejo de sesiones y usuarios

El manejo de sesiones y usuarios constituye un aspecto fundamental de la seguridad y funcionalidad de las aplicaciones web, permitiendo identificar a los usuarios de manera única y mantener su contexto a través de múltiples interacciones con el sistema. La implementación de mecanismos robustos de autenticación y autorización garantiza que solo usuarios legítimos puedan acceder a recursos protegidos y que cada usuario solo pueda realizar acciones para las cuales tiene permisos [2], [3]. Spring Security, integrado con Spring Boot, proporciona un framework comprehensivo para implementar estas funcionalidades de manera declarativa y configurable [17], [22].

4.1. Concepto de sesión

Una sesión representa una secuencia de interacciones relacionadas entre un usuario específico y una aplicación web, manteniendo un estado persistente a través de múltiples solicitudes HTTP. Dado que HTTP es un protocolo stateless que no retiene información entre solicitudes, las sesiones implementan mecanismos para asociar múltiples peticiones

con un mismo usuario, típicamente mediante cookies que contienen identificadores únicos de sesión [17], [21]. Las sesiones modernas pueden implementarse mediante tokens JWT (JSON Web Tokens) que eliminan la necesidad de almacenamiento en el servidor, permitiendo arquitecturas stateless más escalables [23], [24]. El ciclo de vida de una sesión incluye su creación durante el login del usuario, su mantenimiento activo mientras el usuario interactúa con la aplicación, y su destrucción durante el logout o después de un período de inactividad [21], [22].

4.2. Persistencia de información por usuario

La persistencia de información por usuario permite que cada usuario mantenga su propio estado independiente dentro de la aplicación, incluyendo preferencias personalizadas, datos de perfil, historial de actividades y configuraciones específicas. En arquitecturas basadas en sesiones tradicionales, esta información se almacena en memoria del servidor o en bases de datos de sesiones, mientras que en arquitecturas stateless con JWT, la información esencial se codifica en el token mismo [23], [25]. Spring Boot facilita la gestión de datos de usuario mediante el uso de `@SessionAttributes` para mantener objetos en la sesión HTTP y mediante la inyección automática de información del usuario autenticado en los controladores [2], [6]. Es importante distinguir entre datos de sesión temporales que se pierden al finalizar la sesión, y datos persistentes del usuario que deben almacenarse en la base de datos para estar disponibles en sesiones futuras [17], [21].

4.3. Autenticación y control de acceso

La autenticación es el proceso de verificar la identidad de un usuario, típicamente mediante credenciales como nombre de usuario y contraseña, mientras que la autorización determina qué acciones puede realizar un usuario autenticado [2], [3]. Spring Security proporciona soporte integral para implementar tanto autenticación como autorización mediante configuraciones declarativas y anotaciones como `@PreAuthorize` y `@Secured` [2], [6]. Los sistemas modernos implementan control de acceso basado en roles (RBAC) donde los usuarios se asignan a roles específicos (administrador, usuario, visitante) y los permisos se configuran a nivel de rol en lugar de a nivel de usuario individual [23], [26]. La integración de JWT con Spring Security permite implementar autenticación stateless donde cada solicitud incluye un token firmado que contiene la identidad del usuario y sus roles, eliminando la necesidad de consultar la base de datos en cada petición [24], [27].

```

1 @Configuration
2 @EnableWebSecurity
3 public class SecurityConfig {
4
5     @Bean
6     public SecurityFilterChain filterChain(
7         HttpSecurity http) throws Exception {
8
9         http.authorizeHttpRequests(auth -> auth
10             .requestMatchers("/api/public/**").permitAll()
11             .requestMatchers("/api/admin/**")
12                 .hasRole("ADMIN")
13             .requestMatchers("/api/user/**")
14                 .hasAnyRole("USER", "ADMIN")

```

```
15         .anyRequest().authenticated()
16     )
17     .sessionManagement(session -> session
18         .sessionCreationPolicy(
19             SessionCreationPolicy.STATELESS)
20     )
21     .csrf(csrf -> csrf.disable());
22
23     return http.build();
24 }
25 }
26
27 @RestController
28 @RequestMapping("/api/admin")
29 public class AdminController {
30
31     @PreAuthorize("hasRole('ADMIN')")
32     @GetMapping("/users")
33     public List<User> getAllUsers() {
34         return userService.findAll();
35     }
36 }
```

Listing 5: Ejemplo de autenticación con Spring Security

5. Acceso a datos en aplicaciones web

El acceso a datos constituye una capa crítica en aplicaciones web que media entre la lógica de negocio y los sistemas de almacenamiento persistente, típicamente bases de datos relacionales. La implementación eficiente de esta capa impacta directamente en el rendimiento, escalabilidad y mantenibilidad de la aplicación [11], [12]. Spring Data JPA simplifica significativamente el acceso a datos proporcionando abstracciones de alto nivel que eliminan la necesidad de escribir consultas SQL repetitivas y gestionar manualmente las conexiones a la base de datos [10], [11].

5.1. Operaciones CRUD

CRUD (Create, Read, Update, Delete) representa las cuatro operaciones fundamentales de persistencia que toda aplicación orientada a datos debe implementar. Estas operaciones se mapean directamente a sentencias SQL (INSERT, SELECT, UPDATE, DELETE) y a métodos HTTP en APIs RESTful (POST, GET, PUT/PATCH, DELETE) [18], [19]. Las operaciones CRUD proporcionan un marco estandarizado para interactuar con datos, facilitando la comprensión del código y promoviendo la consistencia en la implementación [12], [13]. Spring Data JPA simplifica la implementación de operaciones CRUD mediante interfaces de repositorio que automáticamente generan las consultas SQL necesarias, eliminando código boilerplate y reduciendo la probabilidad de errores [10], [11].

```
1 @Entity
2 @Table(name = "productos")
3 public class Producto {
4     @Id
5     @GeneratedValue(strategy = GenerationType.IDENTITY)
6     private Long id;
7
8     private String nombre;
9     private Double precio;
10    private Integer stock;
11
12    // Getters y setters
13 }
14
15 @Repository
16 public interface ProductoRepository
17     extends JpaRepository<Producto, Long> {
18
19     // Consultas personalizadas
20     List<Producto> findByPrecioLessThan(Double precio);
21
22     @Query("SELECT p FROM Producto p WHERE p.stock > :minStock")
23     List<Producto> findProductsInStock(
24         @Param("minStock") Integer minStock);
25 }
26
27 @Service
28 public class ProductoService {
29
30     @Autowired
31     private ProductoRepository repository;
32
33     // CREATE
34     public Producto crear(Producto producto) {
35         return repository.save(producto);
36     }
37
38     // READ
39     public Optional<Producto> buscarPorId(Long id) {
40         return repository.findById(id);
41     }
42
43     public List<Producto> listarTodos() {
44         return repository.findAll();
45     }
46
47     // UPDATE
48     public Producto actualizar(Long id, Producto datos) {
49         return repository.findById(id)
50             .map(p -> {
51                 p.setNombre(datos.getNombre());
```

```
52         p.setPrecio(datos.getPrecio());
53         p.setStock(datos.getStock());
54         return repository.save(p);
55     })
56     .orElseThrow(() -> new RuntimeException(
57         "Producto no encontrado"));
58 }
59
60 // DELETE
61 public void eliminar(Long id) {
62     repository.deleteById(id);
63 }
64 }
```

Listing 6: Ejemplo de entidad JPA y operaciones CRUD

5.2. Capas de persistencia

CRUD (Create, Read, Update, Delete) representa las cuatro operaciones fundamentales de persistencia que toda aplicación orientada a datos debe implementar. Estas operaciones se mapean directamente a sentencias SQL (INSERT, SELECT, UPDATE, DELETE) y a métodos HTTP en APIs RESTful (POST, GET, PUT/PATCH, DELETE) [18], [19]. Las operaciones CRUD proporcionan un marco estandarizado para interactuar con datos, facilitando la comprensión del código y promoviendo la consistencia en la implementación [12], [13]. Spring Data JPA simplifica la implementación de operaciones CRUD mediante interfaces de repositorio que automáticamente generan las consultas SQL necesarias, eliminando código boilerplate y reduciendo la probabilidad de errores [10], [11].

5.3. Frameworks de acceso a datos

Los frameworks de mapeo objeto-relacional (ORM) como Hibernate y JPA resuelven el problema de impedancia objeto-relacional, proporcionando un puente entre el paradigma orientado a objetos de Java y el modelo relacional de las bases de datos [10], [11]. Hibernate permite escribir clases persistentes siguiendo principios orientados a objetos naturales, incluyendo herencia y polimorfismo, mientras maneja automáticamente el mapeo a estructuras tabulares relacionales [11], [12]. Spring Data JPA construye sobre JPA añadiendo capacidades adicionales como repositorios automáticos, consultas derivadas de nombres de métodos y soporte para paginación y ordenamiento [10], [11]. Estos frameworks mejoran significativamente la productividad del desarrollo al eliminar la necesidad de escribir código repetitivo de acceso a datos y gestionar manualmente el ciclo de vida de las entidades [2], [6].

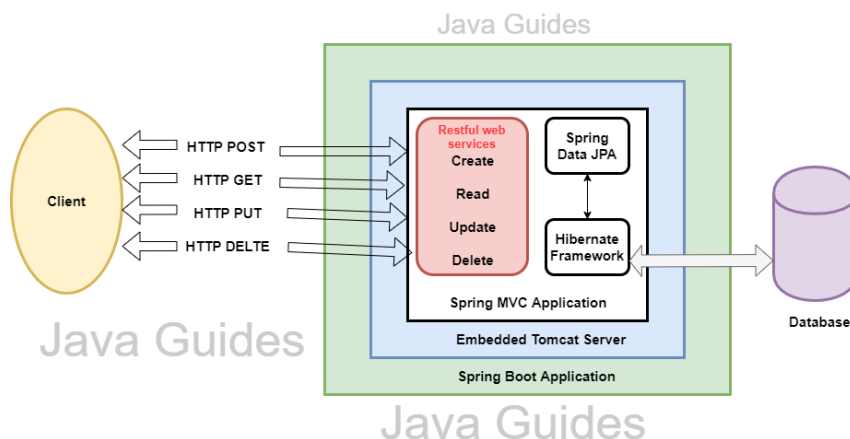


Figura 4: Arquitectura de Spring Data JPA y su integración con Hibernate

6. Procedimientos almacenados

Los procedimientos almacenados son conjuntos precompilados de sentencias SQL almacenadas directamente en el servidor de base de datos, proporcionando una forma eficiente de ejecutar lógica de negocio compleja cerca de los datos. Estos procedimientos pueden aceptar parámetros de entrada, ejecutar múltiples operaciones en una transacción atómica y devolver resultados al código de aplicación [13], [28]. La utilización de procedimientos almacenados representa una estrategia de optimización importante en aplicaciones con operaciones de datos intensivas, particularmente cuando se requiere procesar grandes volúmenes de información [12], [13].

6.1. Definición y características

Un procedimiento almacenado es un programa almacenado en el catálogo de la base de datos que puede ser invocado por aplicaciones cliente, otros procedimientos almacenados o triggers. Los procedimientos almacenados encapsulan lógica de negocio, validaciones, procesamiento condicional y manejo de transacciones directamente en la base de datos [13], [28]. Estos procedimientos se compilan y optimizan una sola vez, almacenando su plan de ejecución en caché para reutilización en invocaciones posteriores, lo que mejora significativamente el rendimiento comparado con consultas ad-hoc [11], [12]. Los procedimientos almacenados soportan variables locales, estructuras de control de flujo (IF, WHILE, CASE), manejo de excepciones y pueden devolver resultados mediante parámetros de salida o conjuntos de resultados [13], [28].

Un ejemplo básico de procedimiento almacenado en PostgreSQL que realiza una operación con validaciones se muestra a continuación:

```

1 CREATE OR REPLACE FUNCTION sp_registrar_producto(
2     p_nombre VARCHAR(100),
3     p_precio DECIMAL(10,2),
4     p_stock INT
5 ) RETURNS TABLE(resultado INT, mensaje VARCHAR) AS $$
6 BEGIN
7     -- Validar que el precio sea positivo
8     IF p_precio <= 0 THEN
9         RETURN QUERY SELECT 0, 'El precio debe ser mayor a cero';
10        RETURN;
11    END IF;
12
13    -- Insertar el nuevo producto
14    INSERT INTO productos (nombre, precio, stock, fecha_creacion)
15    VALUES (p_nombre, p_precio, p_stock, NOW());
16
17    RETURN QUERY SELECT 1, 'Producto registrado exitosamente';
18 END;
19 $$ LANGUAGE plpgsql;

```

Listing 7: Procedimiento almacenado básico en PostgreSQL

6.2. Ventajas y casos de uso

Los procedimientos almacenados ofrecen múltiples ventajas significativas en el desarrollo de aplicaciones orientadas a datos. La reducción del tráfico de red constituye un beneficio principal, ya que en lugar de enviar múltiples consultas SQL, la aplicación invoca el procedimiento una sola vez, ejecutando toda la lógica en el servidor de base de datos [11], [13]. Los procedimientos almacenados mejoran la seguridad al permitir otorgar permisos de ejecución sin exponer el acceso directo a las tablas subyacentes, protegiendo contra inyecciones SQL al usar parámetros en lugar de concatenación de cadenas [12], [28]. El rendimiento mejorado resulta de la precompilación y caché de planes de ejecución, especialmente beneficioso para consultas complejas con múltiples joins y agregaciones [11], [12]. Los casos de uso ideales incluyen procesamiento batch de grandes volúmenes de datos, implementación de lógica de negocio que requiere múltiples operaciones transaccionales y operaciones que manipulan grandes conjuntos de datos donde transferir datos a la aplicación sería ineficiente [13], [28].

Un caso de uso común es el procesamiento de ventas que requiere actualizar múltiples tablas de forma atómica:

```

1 CREATE OR REPLACE FUNCTION sp_procesar_venta(
2     p_producto_id BIGINT,
3     p_cantidad INT,
4     p_usuario_id BIGINT
5 ) RETURNS TABLE(venta_id BIGINT, resultado INT, mensaje VARCHAR)
6 AS $$
7 DECLARE
8     v_stock_actual INT;
9     v_precio DECIMAL(10,2);
10    v_venta_id BIGINT;

```

```

10 BEGIN
11     -- Obtener stock y precio con bloqueo
12     SELECT stock, precio INTO v_stock_actual, v_precio
13     FROM productos WHERE id = p_producto_id FOR UPDATE;
14
15     -- Validar stock disponible
16     IF v_stock_actual < p_cantidad THEN
17         RETURN QUERY SELECT NULL::BIGINT, 0,
18             'Stock insuficiente. Disponible: ' || v_stock_actual;
19     RETURN;
20 END IF;
21
22     -- Registrar venta
23     INSERT INTO ventas (usuario_id, fecha_venta, total)
24     VALUES (p_usuario_id, NOW(), v_precio * p_cantidad)
25     RETURNING id INTO v_venta_id;
26
27     -- Actualizar stock
28     UPDATE productos
29     SET stock = stock - p_cantidad
30     WHERE id = p_producto_id;
31
32     RETURN QUERY SELECT v_venta_id, 1, 'Venta procesada
33     exitosamente';
34 END;
35 $$ LANGUAGE plpgsql;

```

Listing 8: Procedimiento almacenado con transacciones para procesar ventas

6.3. Integración con aplicaciones web

Spring Data JPA facilita la integración de procedimientos almacenados mediante la anotación `@Procedure` que permite invocar procedimientos desde interfaces de repositorio de manera declarativa. Los desarrolladores pueden mapear métodos de repositorio directamente a procedimientos almacenados, especificando el nombre del procedimiento y los parámetros de entrada y salida [10], [11]. La integración con procedimientos almacenados requiere consideraciones cuidadosas sobre dónde ubicar la lógica de negocio, balanceando entre centralización en la base de datos y mantenimiento de código en la aplicación [12], [13]. Los equipos deben adoptar prácticas como versionamiento de procedimientos almacenados, documentación clara de su comportamiento y herramientas de migración como Flyway o Liquibase para gestionar cambios en la lógica de base de datos [2], [6].

Para invocar procedimientos almacenados desde Spring Boot, primero se define el procedimiento en PostgreSQL:


```

1 CREATE OR REPLACE FUNCTION sp_buscar_productos_disponibles(
2     p_precio_minimo DECIMAL(10,2),
3     p_stock_minimo INT
4 ) RETURNS TABLE(
5     id BIGINT,
6     nombre VARCHAR,
7     precio DECIMAL,
8     stock INT
9 ) AS $$
10 BEGIN
11     RETURN QUERY
12     SELECT p.id, p.nombre, p.precio, p.stock
13     FROM productos p
14     WHERE p.precio >= p_precio_minimo
15           AND p.stock >= p_stock_minimo
16           AND p.activo = TRUE
17     ORDER BY p.stock DESC;
18 END;
19 $$ LANGUAGE plpgsql;

```

Listing 9: Procedimiento almacenado para búsqueda de productos

La invocación desde Spring Boot se realiza mediante la configuración del repositorio:

```

1 @Repository
2 public interface ProductoRepository
3     extends JpaRepository<Producto, Long> {
4
5     @Procedure(name = "sp_buscar_productos_disponibles")
6     List<Producto> buscarProductosDisponibles(
7         @Param("p_precio_minimo") Double precioMinimo,
8         @Param("p_stock_minimo") Integer stockMinimo
9     );
10 }
11
12 @Service
13 public class ProductoService {
14     @Autowired
15     private ProductoRepository repository;
16
17     public List<Producto> obtenerProductosEnRango(Double
18 minPrecio) {
19         return repository.buscarProductosDisponibles(minPrecio,
20 10);
21     }
22 }

```

Listing 10: Integración del procedimiento con Spring Data JPA

7. Buenas prácticas en la gestión de información

La implementación de buenas prácticas en la gestión de información es fundamental para construir aplicaciones web robustas, seguras y mantenibles. Estas prácticas abarcan desde decisiones arquitectónicas sobre el ciclo de vida de los datos hasta consideraciones de seguridad y separación de responsabilidades [2], [3]. El cumplimiento de estas prácticas reduce la deuda técnica, facilita la evolución del software y minimiza vulnerabilidades de seguridad [11], [12].

7.1. Alcance y ciclo de vida de los datos

La gestión apropiada del alcance y ciclo de vida de los datos requiere comprender cuándo y dónde almacenar información según su naturaleza y requisitos de persistencia. Los datos de alcance de request deben usarse para información transitoria que solo es relevante durante el procesamiento de una solicitud individual [18], [20]. Los datos de sesión son apropiados para mantener el estado del usuario a través de múltiples solicitudes, pero deben limitarse a información no crítica y con tiempos de expiración razonables para evitar consumo excesivo de memoria [17], [21]. Los datos que necesitan persistir más allá de la sesión del usuario deben almacenarse en la base de datos mediante la capa de persistencia, asegurando su disponibilidad en sesiones futuras [10], [11]. El uso inadecuado de ámbitos de almacenamiento puede resultar en fugas de memoria, problemas de concurrencia y comportamiento inconsistente de la aplicación [12], [13].

7.2. Seguridad de la información

La seguridad de la información debe considerarse en todos los aspectos de la gestión de datos, desde la autenticación y autorización hasta el cifrado de datos en tránsito y en reposo. Las aplicaciones deben implementar validación rigurosa de entradas para prevenir inyecciones SQL, XSS y otros ataques de inyección [2], [3]. Las contraseñas nunca deben almacenarse en texto plano, requiriendo el uso de algoritmos de hashing seguros como bcrypt o Argon2 con sales únicas por usuario [23], [26]. Los datos sensibles transmitidos entre cliente y servidor deben protegerse mediante HTTPS, y las sesiones deben implementar regeneración de identificadores después de autenticación exitosa para prevenir fijación de sesión [17], [22]. Spring Security proporciona protecciones integradas contra muchas vulnerabilidades comunes cuando se configura correctamente [2], [6].

7.3. Separación de responsabilidades

El principio de separación de responsabilidades dicta que diferentes aspectos de la funcionalidad de la aplicación deben manejarse por componentes distintos y especializados. La arquitectura en capas implementa este principio separando la presentación, lógica de negocio y acceso a datos en componentes independientes con responsabilidades claramente definidas [8], [9]. Los controladores deben limitarse a manejar solicitudes HTTP y delegarla lógica de negocio a servicios, los servicios deben implementar reglas de negocio sin preocuparse por detalles de presentación o persistencia, y los repositorios deben enfocarse exclusivamente en operaciones de acceso a datos [2], [6]. Esta separación facilita las pruebas unitarias al permitir mockear dependencias, mejora la mantenibilidad al localizar cambios en componentes específicos, y promueve la reutilización de código al evitar duplicación de lógica [1], [9].

Desarrollo de una aplicación web para la gestión de información

8. Descripción del entorno de desarrollo

La implementación del sistema de gestión de productos con roles requirió establecer un entorno de desarrollo integrando herramientas modernas para el desarrollo de aplicaciones web. Esta sección detalla las tecnologías seleccionadas, su configuración y las herramientas utilizadas durante el ciclo de desarrollo, justificando cada elección en función de los requisitos de la práctica.

8.1. Lenguaje y framework

El desarrollo utiliza Java como lenguaje base y Spring Boot como framework principal. Spring Boot simplifica la configuración mediante anotaciones y convenciones, integrando módulos como Spring Web para APIs REST, Spring Data JPA para persistencia y Spring Security para autenticación basada en roles.

```

1 <dependencies>
2   <!-- Spring Boot Starter Web -->
3   <dependency>
4     <groupId>org.springframework.boot</groupId>
5     <artifactId>spring-boot-starter-web</artifactId>
6   </dependency>
7
8   <!-- Spring Boot Starter Data JPA -->
9   <dependency>
10    <groupId>org.springframework.boot</groupId>
11    <artifactId>spring-boot-starter-data-jpa</artifactId>
12  </dependency>
13
14  <!-- PostgreSQL Driver -->
15  <dependency>
16    <groupId>org.postgresql</groupId>
17    <artifactId>postgresql</artifactId>
18    <scope>runtime</scope>
19  </dependency>
20
21  <!-- Lombok -->
22  <dependency>
23    <groupId>org.projectlombok</groupId>
24    <artifactId>lombok</artifactId>
25    <optional>true</optional>
26  </dependency>
27
28  <!-- ModelMapper -->
29  <dependency>
30    <groupId>org.modelmapper</groupId>
31    <artifactId>modelmapper</artifactId>
32    <version>3.1.1</version>
33  </dependency>
34
35  <!-- Spring Security -->
36  <dependency>
37    <groupId>org.springframework.boot</groupId>

```

```

38     <artifactId>spring-boot-starter-security</artifactId>
39     </dependency>
40 </dependencies>

```

Listing 11: Dependencias principales del proyecto (pom.xml)

La arquitectura sigue el patrón MVC mediante capas: controladores (manejo de HTTP), servicios (lógica de negocio) y repositorios (acceso a datos).

8.2. Sistema gestor de base de datos

Se utiliza PostgreSQL como SGBD relacional. La conexión se configura mediante el driver JDBC postgresql.

```

1 spring.application.name=practicaIV
2 spring.jpa.database=postgresql
3 server.port=8080
4
5 # Configuración JPA/Hibernate
6 spring.jpa.show-sql=true
7 spring.jpa.hibernate.ddl-auto=update
8 spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect
9
10 # Configuración de datasource
11 spring.datasource.driver-class-name=org.postgresql.Driver
12 spring.datasource.url=jdbc:postgresql://localhost:5432/practica_iv
13 spring.datasource.username=postgres
14 spring.datasource.password=12345
15
16 # Configuración de logs
17 logging.level.org.hibernate.SQL=DEBUG
18 logging.level.org.hibernate.type.descriptor.sql.BasicBinder=TRACE

```

Listing 12: Configuración de conexión a PostgreSQL (application.properties)

PostgreSQL permite implementar procedimientos almacenados en PL/pgSQL para lógica compleja y operaciones transaccionales eficientes.

8.3. Herramientas de desarrollo

- **IntelliJ IDEA:** IDE con soporte completo para Spring Boot, autocompletado inteligente y herramientas de refactorización.
- **Maven:** Gestión de dependencias y ciclo de vida del proyecto mediante el archivo pom.xml.
- **Postman:** Testing de endpoints REST, validación de respuestas y gestión de colecciones de pruebas.
- **pgAdmin 4:** Administración gráfica de PostgreSQL para gestión de bases de datos y ejecución de scripts SQL.

9. Diseño general de la aplicación

El diseño del sistema se fundamenta en una arquitectura en capas que separa responsabilidades y facilita el mantenimiento y escalabilidad. Esta arquitectura implementa el patrón MVC adaptado para APIs RESTful, donde los controladores manejan peticiones HTTP, los servicios encapsulan la lógica de negocio, y los repositorios abstraen el acceso a datos. El modelo de datos se compone de cuatro entidades principales relacionadas que representan el dominio del negocio: clientes, productos, ventas y usuarios del sistema.

9.1. Estructura del sistema

El sistema sigue una arquitectura en capas basada en Spring Boot, organizando el código en paquetes con responsabilidades claramente definidas:

- **config:** Configuraciones de Spring (beans, seguridad, ModelMapper)
- **controllers:** Endpoints REST que manejan peticiones HTTP y construyen respuestas
- **dto:** Objetos de transferencia de datos para comunicación cliente-servidor
- **entities:** Entidades JPA que mapean tablas de la base de datos
- **repositories:** Interfaces de acceso a datos basadas en Spring Data JPA
- **services:** Lógica de negocio, validaciones y orquestación de operaciones

El flujo de información sigue el patrón: **Cliente HTTP** → **Controller** → **Service** → **Repository** → **Base de Datos**, retornando los resultados en sentido inverso. Esta separación permite que cada capa sea testeable de forma independiente y facilita la evolución del sistema sin afectar otros componentes.

9.2. Entidades principales

El modelo de datos está compuesto por cuatro entidades principales que representan el dominio del negocio:

9.2.1. Client (Cliente)

Representa a los clientes del sistema que realizan compras. Incluye información de contacto y un RUC único como identificador fiscal.

```

1 @Entity
2 @Table(name = "client")
3 @Data
4 @AllArgsConstructor
5 @NoArgsConstructor
6 public class Client {
7     @Id
8     @GeneratedValue(strategy = GenerationType.IDENTITY)
9     private Long id;
10
11     @Column(name = "ruc", unique = true, nullable = false, length = 13)
12     private String ruc;

```

```

13
14     @Column(name = "name", nullable = false, length = 50)
15     private String name;
16
17     @Column(name = "address", nullable = false, length = 80)
18     private String address;
19
20     @Column(name = "email", nullable = false, length = 80)
21     private String email;
22
23     @Column(name = "phone", nullable = false, length = 80)
24     private String phone;
25 }

```

Listing 13: Entidad Client

9.2.2. Product (Producto)

Representa los productos disponibles para la venta, incluyendo información de precio, stock y descripción.

```

1 @Entity
2 @Table(name = "product")
3 @Getter
4 @Setter
5 @AllArgsConstructor
6 @NoArgsConstructor
7 public class Product {
8     @Id
9     @GeneratedValue(strategy = GenerationType.IDENTITY)
10    private Long codigo;
11
12    @Column(name = "nombre", nullable = false, length = 50, unique =
13    true)
14    private String nombre;
15
16    @Column(name = "precio", nullable = false,
17            columnDefinition = "decimal(9,2)")
18    private Double precio;
19
20    @Column(name = "stock", nullable = false)
21    private Integer stock = 0;
22
23    @Column(name = "descripcion", length = 200)
24    private String descripcion;
25 }

```

Listing 14: Entidad Product

9.2.3. Sale (Venta)

Representa una transacción de venta realizada por un cliente, incluyendo fecha y total de la operación. Mantiene una relación OneToMany con SaleDetail.

```

1 @Entity
2 @Table(name = "sale")
3 @Data
4 @AllArgsConstructor

```

```

5 @NoArgsConstructor
6 @EqualsAndHashCode(onlyExplicitlyIncluded = true)
7 public class Sale {
8     @Id
9     @GeneratedValue(strategy = GenerationType.IDENTITY)
10    @EqualsAndHashCode.Include
11    private Long number;
12
13    @ManyToOne
14    @JoinColumn(name = "client_id", nullable = false,
15                foreignKey = @ForeignKey(name = "fk_sale_client"))
16    private Client client;
17
18    @Column(name = "sale_date", nullable = false)
19    private LocalDateTime saleDate;
20
21    @Column(name = "total", nullable = false,
22            columnDefinition = "decimal(10,2)")
23    private Double total;
24
25    @OneToMany(mappedBy = "sale", cascade = CascadeType.ALL,
26              orphanRemoval = true)
27    private List<SaleDetail> details = new ArrayList<>();
28
29    // Metodo helper para gestionar relacion bidireccional
30    public void addDetail(SaleDetail detail) {
31        details.add(detail);
32        detail.setSale(this);
33    }
34 }

```

Listing 15: Entidad Sale con relaciones

9.2.4. SaleDetail (Detalle de Venta)

Representa cada línea de producto vendido dentro de una venta, incluyendo cantidad, precio unitario al momento de la venta y descuentos aplicados.

```

1 @Entity
2 @Table(name = "sale_detail")
3 @Data
4 @AllArgsConstructor
5 @NoArgsConstructor
6 @EqualsAndHashCode(onlyExplicitlyIncluded = true)
7 public class SaleDetail {
8     @Id
9     @GeneratedValue(strategy = GenerationType.IDENTITY)
10    @EqualsAndHashCode.Include
11    private Long id;
12
13    @ManyToOne
14    @JoinColumn(name = "sale_id", nullable = false,
15                foreignKey = @ForeignKey(name = "fk_detail_sale"))
16    private Sale sale;
17
18    @ManyToOne
19    @JoinColumn(name = "product_id", nullable = false,
20                foreignKey = @ForeignKey(name = "fk_detail_product"))

```

```

21     private Product product;
22
23     @Column(nullable = false, name = "quantity")
24     private Integer quantity;
25
26     @Column(name = "price", nullable = false,
27             columnDefinition = "decimal(9,2)")
28     private Double price;
29
30     @Column(name = "discount",
31             columnDefinition = "decimal(9,2) default 0.00")
32     private Double discount = 0.0;
33
34     @Transient
35     public Double getSubtotal() {
36         return (price * quantity) - discount;
37     }
38 }

```

Listing 16: Entidad SaleDetail

9.2.5. User (Usuario del Sistema)

Representa los usuarios que pueden autenticarse en el sistema, con roles diferenciados para control de acceso.

```

1  @Entity
2  @Table(name = "users")
3  @Data
4  @AllArgsConstructor
5  @NoArgsConstructor
6  public class User {
7      @Id
8      @GeneratedValue(strategy = GenerationType.IDENTITY)
9      private Long id;
10
11      @Column(nullable = false, unique = true, length = 50)
12      private String username;
13
14      @Column(nullable = false)
15      private String password;
16
17      @Column(nullable = false, length = 100)
18      private String fullName;
19
20      @Column(nullable = false, length = 20)
21      @Enumerated(EnumType.STRING)
22      private Role role;
23
24      @Column(nullable = false)
25      private Boolean active = true;
26
27      private LocalDateTime createdAt = LocalDateTime.now();
28  }
29
30  public enum Role {
31      ADMIN,
32      VENDEDOR,

```


33 CLIENTE
34 }

Listing 17: Entidad User y enum Role

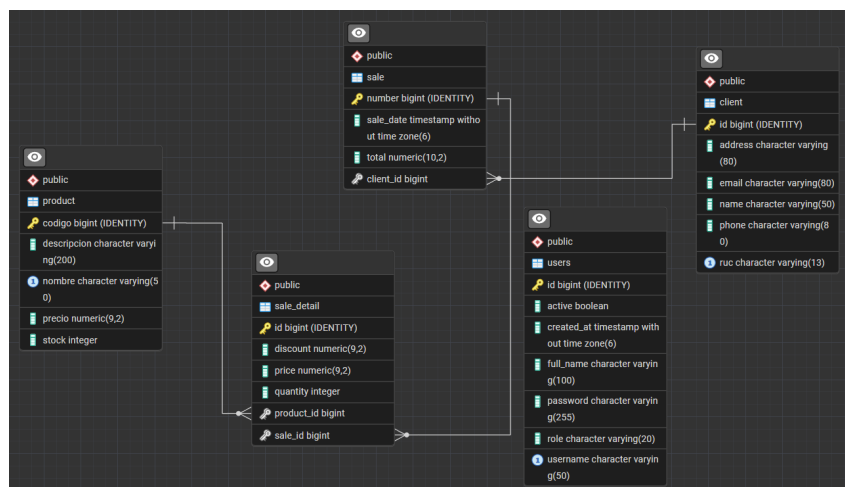


Figura 5: Diagrama Entidad-Relación de la base de datos

9.3. Roles de usuario

El sistema implementa tres roles diferenciados mediante Spring Security para controlar el acceso a las funcionalidades según el tipo de usuario:

Cuadro 1: Roles de usuario y permisos del sistema

Rol	Permisos	Descripción
ADMIN	- CRUD completo de productos - CRUD de clientes - Ver todas las ventas - Generar reportes - Gestionar usuarios	Administrador con acceso total al sistema. Puede realizar todas las operaciones sin restricciones.
VENDEDOR	- Crear ventas - Consultar productos - Consultar clientes - Ver sus propias ventas - Consultar total del día	Usuario encargado del proceso de ventas. Tiene permisos limitados enfocados en operaciones comerciales.
CLIENTE	- Ver catálogo de productos - Ver su historial de compras - Actualizar sus datos personales	Cliente del sistema con acceso de solo lectura a productos y acceso limitado a sus propios datos.

La implementación de roles se realiza mediante anotaciones de Spring Security en los controladores, validando permisos antes de ejecutar cada operación. El control de acceso basado en roles (RBAC) garantiza que cada usuario solo pueda realizar acciones autorizadas según su nivel de privilegios.

10. Implementación de la gestión de información

La gestión de información en aplicaciones web multiusuario requiere mecanismos sofisticados para capturar datos de las peticiones, construir respuestas apropiadas y mantener estado entre múltiples interacciones. Esta sección detalla la implementación práctica de los objetos del ambiente web (Request, Response, Session) mediante controladores REST de Spring Boot, demostrando cómo cada objeto contribuye al flujo de información y al control de acceso en el sistema.

10.1. Manejo de Request y Response

El manejo de objetos Request y Response se implementa mediante anotaciones de Spring Boot en los controladores REST, permitiendo capturar parámetros, headers y construir respuestas personalizadas con códigos de estado apropiados.

10.1.1. Captura de parámetros del Request

El siguiente ejemplo demuestra cómo capturar diferentes tipos de información del Request: parámetros de consulta, headers HTTP y metadatos de la petición.

```

1 @RestController
2 @RequestMapping("/api/products")
3 @RequiredArgsConstructor
4 public class ProductController {
5
6     private final IProductService productService;
7     private final ModelMapper modelMapper;
8
9     @GetMapping("/search")
10    public ResponseEntity<Map<String, Object>> searchProducts(
11        @RequestParam(required = false) String nombre,
12        @RequestParam(required = false) Double minPrice,
13        @RequestParam(required = false) Double maxPrice,
14        @RequestHeader(value = "User-Agent", required = false)
15            String userAgent,
16        HttpServletRequest request) {
17
18        Map<String, Object> response = new HashMap<>();
19
20        // Captura de informacion del Request
21        response.put("requestInfo", Map.of(
22            "method", request.getMethod(),
23            "uri", request.getRequestURI(),
24            "userAgent", userAgent != null ? userAgent : "Unknown",
25            "remoteIP", request.getRemoteAddr()
26        ));
27
28        // Parametros de busqueda
29        response.put("searchParams", Map.of(
30            "nombre", nombre != null ? nombre : "N/A",
31            "minPrice", minPrice != null ? minPrice : 0,
32            "maxPrice", maxPrice != null ? maxPrice : "Sin limite"
33        ));
34
35        // Ejecutar busqueda
36        List<ProductDTO> products = productService

```

```

37         .searchProducts(nombre, minPrice, maxPrice)
38         .stream()
39         .map(this::convertToDTO)
40         .toList();
41
42     response.put("results", products);
43     response.put("count", products.size());
44     response.put("timestamp", LocalDateTime.now());
45
46     return ResponseEntity.ok(response);
47 }
48 }

```

Listing 18: ProductController - Captura de Request params y headers

Puntos clave del código:

- `@RequestParam`: Captura parámetros de consulta (query params) con soporte para valores opcionales
- `@RequestHeader`: Accede a encabezados HTTP específicos como User-Agent
- `HttpServletRequest`: Objeto completo con metadatos de la petición (método, URI, IP del cliente)
- La respuesta incluye tanto los resultados de búsqueda como información contextual del Request

10.1.2. Personalización del Response

El siguiente ejemplo demuestra cómo construir respuestas HTTP personalizadas con diferentes códigos de estado y headers según el resultado de la operación.

```

1 @PostMapping
2 public ResponseEntity<Map<String, Object>> createProduct(
3     @RequestBody ProductDTO productDTO,
4     HttpServletRequest request) {
5
6     Map<String, Object> response = new HashMap<>();
7
8     try {
9         Product product = productService.save(
10             convertToEntity(productDTO)
11         );
12
13         // Response exitoso (201 Created)
14         response.put("success", true);
15         response.put("message", "Producto creado exitosamente");
16         response.put("data", convertToDTO(product));
17         response.put("timestamp", LocalDateTime.now());
18
19         return ResponseEntity
20             .status(HttpStatus.CREATED)
21             .header("X-Created-ID", product.getCodigo().toString())
22             .header("Location", "/api/products/" + product.getCodigo()
23             .body(response);
24

```

```

25     } catch (Exception e) {
26         // Response de error (400 Bad Request)
27         response.put("success", false);
28         response.put("message", "Error: " + e.getMessage());
29         response.put("timestamp", LocalDateTime.now());
30
31         return ResponseEntity
32             .status(HttpStatus.BAD_REQUEST)
33             .body(response);
34     }
35 }
36
37 @GetMapping("/{codigo}")
38 public ResponseEntity<ProductDTO> getProduct(@PathVariable Long codigo)
39 {
40     try {
41         Product product = productService.findById(codigo);
42         return ResponseEntity
43             .ok()
44             .header("X-Product-Stock", product.getStock().toString())
45             .body(convertToDTO(product));
46     } catch (Exception e) {
47         return ResponseEntity.notFound().build(); // 404
48     }
49 }
50
51 @DeleteMapping("/{codigo}")
52 public ResponseEntity<Void> deleteProduct(@PathVariable Long codigo) {
53     try {
54         productService.delete(codigo);
55         return ResponseEntity.noContent().build(); // 204 No Content
56     } catch (Exception e) {
57         return ResponseEntity.notFound().build(); // 404
58     }
59 }

```

Listing 19: Response personalizado con códigos de estado y headers

Códigos de estado HTTP utilizados:

- **200 OK:** Operación exitosa con contenido en el cuerpo
- **201 Created:** Recurso creado exitosamente
- **204 No Content:** Operación exitosa sin contenido en el cuerpo (DELETE)
- **400 Bad Request:** Error de validación o datos inválidos
- **404 Not Found:** Recurso no encontrado

10.2. Gestión de sesiones

La gestión de sesiones permite mantener información del usuario autenticado a través de múltiples peticiones HTTP, implementando control de acceso basado en roles y persistencia de estado.

10.2.1. Configuración de Spring Security

```

1 @Configuration
2 public class SecurityConfig {
3
4     @Bean
5     public SecurityFilterChain filterChain(HttpSecurity http)
6         throws Exception {
7         http
8             .csrf(csrf -> csrf.disable())
9             .cors(cors -> cors.configurationSource(
10                 corsConfigurationSource()))
11             .authorizeHttpRequests(auth -> auth
12                 .anyRequest().permitAll()
13             );
14
15         return http.build();
16     }
17
18     @Bean
19     public PasswordEncoder passwordEncoder() {
20         return new BCryptPasswordEncoder();
21     }
22 }

```

Listing 20: Configuración de seguridad y gestión de sesiones

10.2.2. Autenticación y creación de sesión

```

1 @RestController
2 @RequestMapping("/api/auth")
3 @RequiredArgsConstructor
4 public class AuthController {
5
6     private final IUserService userService;
7
8     @PostMapping("/login")
9     public ResponseEntity<Map<String, Object>> login(
10         @RequestBody LoginRequest request,
11         HttpSession session) throws Exception {
12
13         User user = userService.authenticate(
14             request.getUsername(),
15             request.getPassword()
16         );
17         //Almacenar datos en la sesion
18         session.setAttribute("userId", user.getId());
19         session.setAttribute("username", user.getUsername());
20         session.setAttribute("role", user.getRole().toString());
21
22         return ResponseEntity.ok(Map.of(
23             "success", true,
24             "message", "Login exitoso"
25         ));
26     }
27 }

```

Listing 21: AuthController - Login y creación de sesión

10.2.3. Consulta de información de sesión

```

1 @GetMapping("/session-info")
2 public ResponseEntity<Map<String, Object>> getSessionInfo(
3     HttpSession session) {
4
5     Map<String, Object> response = new HashMap<>();
6
7     String username = (String) session.getAttribute("username");
8
9     if (username == null) {
10         response.put("authenticated", false);
11         response.put("message", "No hay sesión activa");
12         return ResponseEntity.status(HttpStatus.UNAUTHORIZED)
13             .body(response);
14     }
15
16     response.put("authenticated", true);
17     response.put("sessionId", session.getId());
18     response.put("username", username);
19     response.put("fullName", session.getAttribute("fullName"));
20     response.put("role", session.getAttribute("role"));
21     response.put("loginTime", session.getAttribute("loginTime"));
22     response.put("lastAccessedTime",
23         Instant.ofEpochMilli(session.getLastAccessedTime()));
24     response.put("maxInactiveInterval",
25         session.getMaxInactiveInterval() + " segundos");
26
27     return ResponseEntity.ok(response);
28 }
29
30 @PostMapping("/logout")
31 public ResponseEntity<Map<String, Object>> logout(HttpSession session) {
32
33     Map<String, Object> response = new HashMap<>();
34
35     String username = (String) session.getAttribute("username");
36     session.invalidate(); // Destruir sesión
37
38     response.put("success", true);
39     response.put("message", "Sesión cerrada exitosamente");
40     response.put("user", username);
41
42     return ResponseEntity.ok(response);
43 }

```

Listing 22: Recuperar información almacenada en la sesión

10.2.4. Uso de sesión en operaciones protegidas

```

1 @RestController
2 @RequestMapping("/api/sales")
3 @RequiredArgsConstructor
4 public class SaleController {
5
6     private final ISaleService saleService;
7

```

```

8      @PostMapping
9      public ResponseEntity<Map<String, Object>> createSale(
10          @RequestBody SaleDTO saleDTO,
11          HttpSession session) {
12
13          Map<String, Object> response = new HashMap<>();
14
15          // Verificar autenticacion
16          String username = (String) session.getAttribute("username");
17          String role = (String) session.getAttribute("role");
18
19          if (username == null) {
20              response.put("success", false);
21              response.put("message", "Debe iniciar sesion");
22              return ResponseEntity.status(HttpStatus.UNAUTHORIZED)
23                  .body(response);
24          }
25
26          // Verificar permisos (solo ADMIN y VENDEDOR)
27          if (!"ADMIN".equals(role) && !"VENDEDOR".equals(role)) {
28              response.put("success", false);
29              response.put("message", "No tiene permisos para crear ventas
30          ");
31              return ResponseEntity.status(HttpStatus.FORBIDDEN)
32                  .body(response);
33          }
34
35          try {
36              Sale sale = saleService.createSale(saleDTO, username);
37
38              response.put("success", true);
39              response.put("message", "Venta creada exitosamente");
40              response.put("data", convertToDTO(sale));
41              response.put("createdBy", username);
42
43              return ResponseEntity.status(HttpStatus.CREATED)
44                  .header("X-Sale-Number", sale.getNumber().toString())
45                  .body(response);
46          } catch (Exception e) {
47              response.put("success", false);
48              response.put("message", "Error: " + e.getMessage());
49              return ResponseEntity.status(HttpStatus.BAD_REQUEST)
50                  .body(response);
51          }
52      }
53 }

```

Listing 23: SaleController - Validación de sesión y roles

10.3. Flujo de información

El flujo completo de información entre cliente y servidor sigue un ciclo bien definido que involucra los objetos Request, Response y Session:

1. **Cliente envía Request:** El cliente (Postman, navegador) construye una petición HTTP con método, URL, headers, parámetros y cuerpo según sea necesario.

2. **Servidor procesa Request:** El controlador de Spring Boot captura la petición mediante anotaciones (`@GetMapping`, `@PostMapping`, etc.), extrae parámetros con `@RequestParam` y `@RequestBody`, y valida la sesión si es necesaria.
3. **Validación de Session:** Para operaciones protegidas, el servidor verifica que exista una sesión activa mediante `session.getAttribute()`, comprueba el rol del usuario y autoriza o rechaza la operación.
4. **Ejecución de lógica de negocio:** El controlador delega la operación a la capa de servicios, que ejecuta validaciones, consulta o modifica datos en la base de datos mediante repositorios, y aplica reglas de negocio.
5. **Construcción del Response:** El servidor construye una respuesta mediante `ResponseEntity`, estableciendo código de estado HTTP apropiado (200, 201, 400, 401, 403, 404), agregando headers personalizados y serializando el cuerpo de la respuesta a JSON.
6. **Cliente recibe Response:** El cliente procesa la respuesta, extrae el código de estado para determinar el resultado, lee headers para obtener metadatos, y deserializa el cuerpo JSON para presentar la información al usuario.

Ejemplo de flujo completo:

1. Request Login:
 POST /api/auth/login
 username=admin&password=admin123
2. Response Login:
 HTTP/1.1 200 OK
 Set-Cookie: JSESSIONID=ABC123...
 {"success": true, "sessionId": "ABC123", "user": {...}}
3. Request Crear Venta (con sesión):
 POST /api/sales
 Cookie: JSESSIONID=ABC123
 {"clientId": 1, "details": [...]}
4. Servidor valida sesión:
 session.getAttribute("username") → "admin"
 session.getAttribute("role") → "ADMIN"
5. Response Crear Venta:
 HTTP/1.1 201 Created
 X-Sale-Number: 15
 {"success": true, "saleNumber": 15, "createdBy": "admin"}

Este flujo garantiza que la información se gestione de manera segura, que el estado del usuario persista entre peticiones, y que cada operación retorne respuestas claras y accionables para el cliente.

11. Implementación del acceso a datos

El acceso a datos se implementa mediante Spring Data JPA para operaciones CRUD estándar y procedimientos almacenados para operaciones complejas que requieren alta eficiencia transaccional.

11.1. Repositorios y consultas

Los repositorios extienden `JpaRepository` proporcionando operaciones automáticas y consultas personalizadas.

```

1 @Repository
2 public interface ProductRepository
3     extends JpaRepository<Product, Long> {
4
5     // Metodo derivado - Spring genera la consulta
6     List<Product> findByNombreContainingIgnoreCase(String nombre);
7
8     // JPQL con parametros
9     @Query("SELECT p FROM Product p WHERE p.stock > :minStock")
10    List<Product> findProductsWithStock(
11        @Param("minStock") Integer minStock
12    );
13
14    // Busqueda dinamica
15    @Query("SELECT p FROM Product p WHERE " +
16        "(:nombre IS NULL OR LOWER(p.nombre) LIKE " +
17        "LOWER(CONCAT('%', :nombre, '%'))) AND " +
18        "(:minPrice IS NULL OR p.precio >= :minPrice) AND " +
19        "(:maxPrice IS NULL OR p.precio <= :maxPrice)")
20    List<Product> searchProducts(
21        @Param("nombre") String nombre,
22        @Param("minPrice") Double minPrice,
23        @Param("maxPrice") Double maxPrice
24    );
25 }

```

Listing 24: ProductRepository con consultas personalizadas

11.2. Operaciones CRUD

Las operaciones CRUD se implementan en la capa de servicios con validaciones de negocio.

```

1 @Service
2 @RequiredArgsConstructor
3 public class ProductServiceImpl implements IProductService {
4
5     private final ProductRepository productRepository;
6
7     @Override
8     @Transactional
9     public Product save(Product product) throws Exception {
10        // Validaciones
11        if (product.getPrecio() <= 0) {
12            throw new Exception("El precio debe ser mayor a 0");
13        }
14    }
15 }

```

```

14         return productRepository.save(product);
15     }
16
17     @Override
18     @Transactional(readOnly = true)
19     public Product findById(Long id) throws Exception {
20         return productRepository.findById(id)
21             .orElseThrow(() -> new Exception(
22                 "Producto no encontrado: " + id));
23     }
24
25     @Override
26     @Transactional
27     public Product update(Long id, Product product) throws Exception {
28         Product existing = findById(id);
29         existing.setNombre(product.getNombre());
30         existing.setPrecio(product.getPrecio());
31         existing.setStock(product.getStock());
32         return productRepository.save(existing);
33     }
34
35     @Override
36     @Transactional
37     public void delete(Long id) throws Exception {
38         productRepository.delete(findById(id));
39     }
40 }

```

Listing 25: ProductServiceImpl - Operaciones CRUD

Cuadro 2: Mapeo de operaciones CRUD

CRUD	HTTP	JPA	SQL
CREATE	POST	save()	INSERT INTO product ...
READ	GET	findById()	SELECT * FROM product WHERE ...
UPDATE	PUT	save()	UPDATE product SET ...
DELETE	DELETE	delete()	DELETE FROM product WHERE ...

11.3. Uso de procedimientos almacenados

Los procedimientos almacenados encapsulan lógica compleja reduciendo tráfico de red y garantizando consistencia transaccional.

```

1 CREATE OR REPLACE FUNCTION sp_process_sale(
2     p_client_id BIGINT,
3     p_product_id BIGINT,
4     p_quantity INTEGER,
5     p_discount DECIMAL(9,2) DEFAULT 0.00
6 )
7 RETURNS TABLE(
8     sale_number BIGINT,
9     total_amount DECIMAL(10,2),
10    result_code INTEGER,

```

```

11     result_message VARCHAR(200)
12 )
13 LANGUAGE plpgsql
14 AS $$
15 DECLARE
16     v_stock INTEGER;
17     v_price DECIMAL(9,2);
18     v_sale_id BIGINT;
19 BEGIN
20     -- Validar cliente
21     IF NOT EXISTS (SELECT 1 FROM client WHERE id = p_client_id) THEN
22         RETURN QUERY SELECT NULL::BIGINT, NULL::DECIMAL, 0,
23             'Cliente no encontrado'::VARCHAR;
24     RETURN;
25 END IF;
26
27     -- Obtener stock y precio con bloqueo
28     SELECT stock, precio INTO v_stock, v_price
29     FROM product WHERE codigo = p_product_id FOR UPDATE;
30
31     -- Validar stock
32     IF v_stock < p_quantity THEN
33         RETURN QUERY SELECT NULL::BIGINT, NULL::DECIMAL, 0,
34             ('Stock insuficiente: ' || v_stock)::VARCHAR;
35     RETURN;
36 END IF;
37
38     -- Crear venta
39     INSERT INTO sale (client_id, sale_date, total)
40     VALUES (p_client_id, NOW(),
41         (v_price * p_quantity) - p_discount)
42     RETURNING number INTO v_sale_id;
43
44     -- Crear detalle
45     INSERT INTO sale_detail (sale_id, product_id, quantity,
46         price, discount)
47     VALUES (v_sale_id, p_product_id, p_quantity,
48         v_price, p_discount);
49
50     -- Actualizar stock
51     UPDATE product SET stock = stock - p_quantity
52     WHERE codigo = p_product_id;
53
54     -- Retornar resultado
55     RETURN QUERY SELECT v_sale_id,
56         (v_price * p_quantity) - p_discount, 1,
57         'Venta procesada exitosamente'::VARCHAR;
58 END;
59 $$;

```

Listing 26: Procedimiento sp_process_sale

```

1 @Repository
2 public interface SaleRepository extends JpaRepository<Sale, Long> {
3
4     @Query(value = "SELECT * FROM sp_process_sale(:clientId, " +
5         ":productId, :quantity, :discount)",
6         nativeQuery = true)
7     List<Object[]> processSale(

```

```

8      @Param("clientId") Long clientId,
9      @Param("productId") Long productId,
10     @Param("quantity") Integer quantity,
11     @Param("discount") Double discount
12 );
13 }

```

Listing 27: Integración del procedimiento en SaleRepository

```

1  @Override
2  @Transactional
3  public Map<String, Object> processSaleWithProcedure(
4      Long clientId, Long productId,
5      Integer quantity, Double discount) throws Exception {
6
7      List<Object[]> results = saleRepository.processSale(
8          clientId, productId, quantity, discount != null ? discount : 0.0
9      );
10
11     if (results.isEmpty()) {
12         throw new Exception("Error al procesar venta");
13     }
14
15     Object[] row = results.get(0);
16
17     Map<String, Object> response = new HashMap<>();
18     response.put("saleNumber", row[0]);
19     response.put("totalAmount", row[1]);
20     response.put("resultCode", row[2]);
21     response.put("resultMessage", row[3]);
22
23     if ((Integer) row[2] == 0) {
24         throw new Exception((String) row[3]);
25     }
26
27     return response;
28 }

```

Listing 28: Uso del procedimiento en SaleServiceImpl

Cuadro 3: Comparación: JPA vs Procedimientos Almacenados

Aspecto	JPA	Stored Procedures
Tráfico red	Múltiples queries	Una sola llamada
Rendimiento	Bueno para CRUD simple	Excelente para operaciones complejas
Mantenibilidad	Código centralizado en Java	Lógica en BD y Java
Portabilidad	Alta (cualquier BD)	Baja (dependiente de PostgreSQL)
Uso ideal	CRUD estándar	Operaciones batch, reportes

12. Escenarios de prueba y resultados

Se diseñaron casos de prueba para validar el funcionamiento de los objetos del ambiente web y las operaciones de acceso a datos, ejecutados mediante Postman para verificar el comportamiento del sistema.

12.1. Casos de prueba

Cuadro 4: Casos de prueba implementados

ID	Caso de Prueba	Endpoint	Método	Objetivo
CP-01	Crear producto (Response personalizado)	/api/products	POST	Validar Response códigos y estructura JSON
CP-02	Búsqueda de productos (Request)	/api/products/search	GET	Validar captura de Request params y headers
CP-03	Registro de usuario	/api/auth/register	POST	Crear nuevo usuario en el sistema
CP-04	Login y creación de sesión	/api/auth/login	POST	Crear Session y almacenar datos de usuario
CP-05	Información de sesión activa	/api/auth/session-info	GET	Persistencia de datos en Session
CP-06	Control de acceso sin sesión	/api/sales	POST	Validar rechazo sin autenticación
CP-07	Crear venta con JPA (CRUD completo)	/api/sales	POST	CRUD Create con validación de sesión y rol
CP-08	Logout y destrucción de sesión	/api/auth/logout	POST	Invaldar Session

12.2. Resultados obtenidos

12.2.1. CP-01: Crear producto (Response personalizado)

Request:

```

1 {
2   "nombre": "Laptop Dell XPS 15",
3   "precio": 1299.99,
4   "stock": 10,
5   "descripcion": "Laptop profesional de alto rendimiento"
6 }
```

Listing 29: Body del POST para crear producto

Response:

```

1 {
2   "data": {
3     "codigo": 1,
4     "descripcion": "Laptop profesional de alto rendimiento",
```

```

5     "nombre": "Laptop Dell XPS 15",
6     "precio": 1299.99,
7     "stock": 10
8 },
9 "success": true,
10 "message": "Producto creado exitosamente",
11 "timestamp": "2026-01-24T20:35:37.2208331"
12 }

```

Análisis: La respuesta utiliza código HTTP 201 (Created), incluye headers personalizados y estructura JSON apropiada.

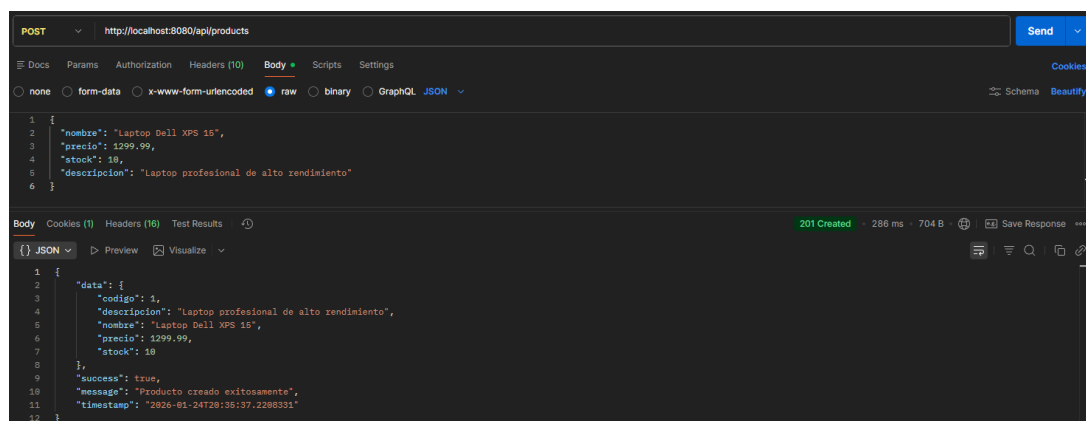


Figura 6: CP-01: Captura de Postman mostrando registro de producto con JSON

12.2.2. CP-02: Búsqueda de productos (Request)

Request ejecutado:

GET `http://localhost:8080/api/products/search`
`?nombre=laptop&minPrice=500&maxPrice=2000`

Response obtenida:

```

1 {
2   "count": 1,
3   "requestInfo": {
4     "method": "GET",
5     "userAgent": "PostmanRuntime/7.51.0",
6     "uri": "/api/products/search",
7     "remoteIP": "0:0:0:0:0:0:0:1"
8   },
9   "results": [
10    {
11      "codigo": 1,
12      "descripcion": "Laptop profesional de alto rendimiento",
13      "nombre": "Laptop Dell XPS 15",
14      "precio": 1299.99,
15      "stock": 10
16    }
17  ],
18  "searchParams": {
19    "maxPrice": 2000.0,
20    "nombre": "laptop",

```

```

21     "minPrice": 500.0
22 },
23     "timestamp": "2026-01-24T20:45:58.2487579"
24 }

```

Listing 30: Response con información del Request

Análisis: El endpoint capturó correctamente los parámetros mediante `@RequestParam` y metadatos del objeto `HttpServletRequest`.

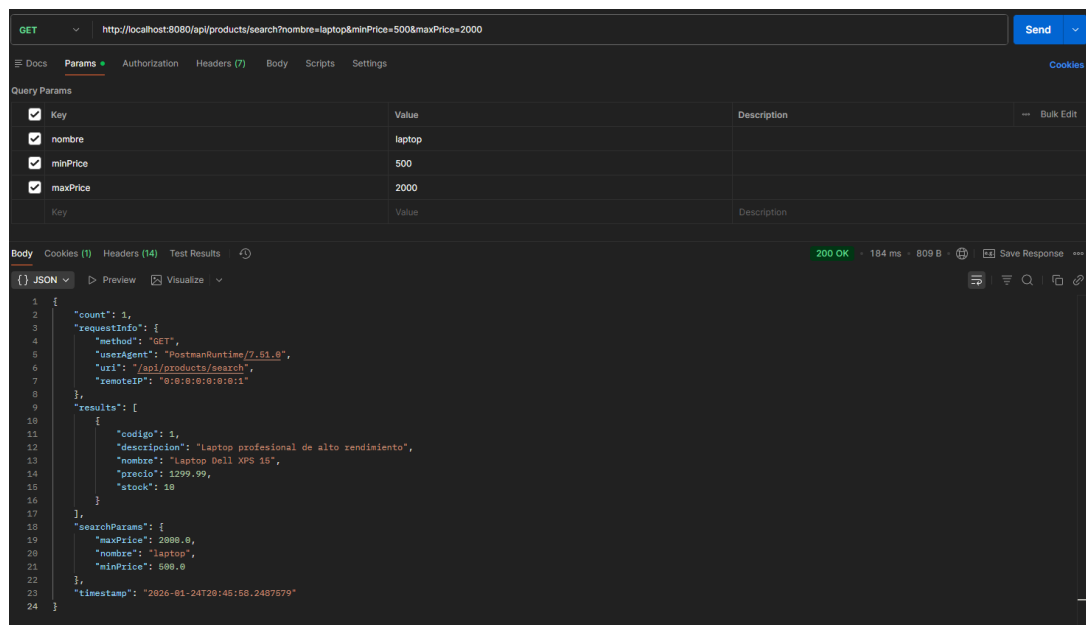


Figura 7: CP-02: Captura de Postman mostrando búsqueda con parámetros

12.2.3. CP-03: Registro de usuario

Request:

POST `http://localhost:8080/api/auth/register`
 Content-Type: `application/json`

```

{
  "username": "eduardo",
  "fullName": "Eduardo",
  "email": "edu@mail.com",
  "password": "1234",
  "role": "ADMIN",
  "active": true
}

```

Response:

```

1 {
2   "success": true,
3   "message": "Usuario creado exitosamente",
4   "user": {
5

```

```

6      "active": true,
7      "email": null,
8      "fullName": "Eduardo",
9      "id": 11,
10     "password": "$2a$10$.XbmIpEjk4FNi3TzzS0Jj.U4t2otLdEyMEhNJJXA/
11                TLSlqNr.rpJS",
12     "role": "ADMIN",
13     "username": "eduardo"
14 }
15 }

```

Listing 31: Response del registro de user

Análisis: Se registró datos de un nuevo usuario.

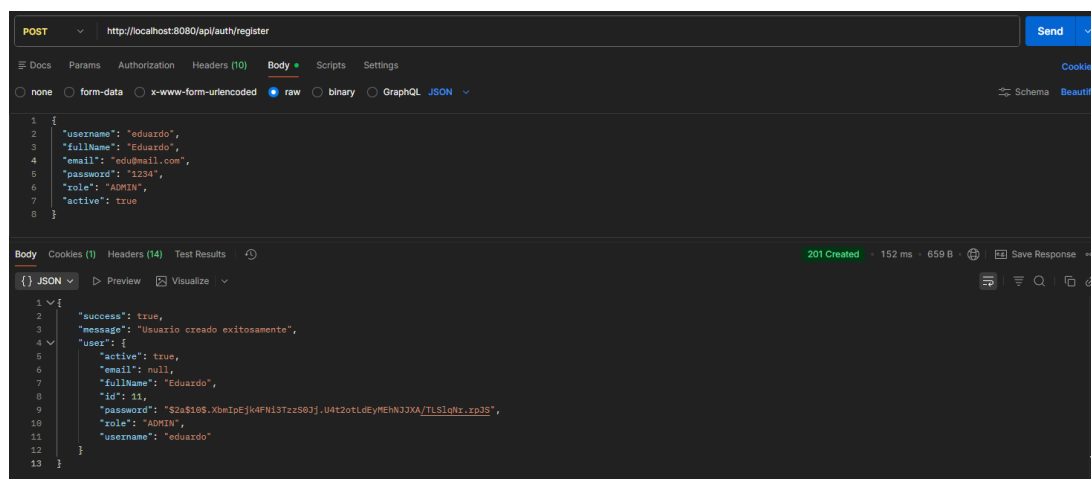


Figura 8: CP-03: Captura de Postman mostrando registro de usuario con JSON

12.2.4. CP-04: Login y creación de sesión

Request:

POST `http://localhost:8080/api/auth/login`

Content-Type: `application/x-www-form-urlencoded`

`username=eduardo&password=1234`

Response:

```

1 {
2   "success": true,
3   "sessionId": "F4465833495DA0434578D2570B96ED68",
4   "message": "Login exitoso",
5   "user": {
6     "fullName": "Eduardo",
7     "username": "eduardo",
8     "role": "ADMIN"
9   }
10 }
11 }

```

Listing 32: Response del login con sessionId

Análisis: El servidor creó una sesión mediante `HttpSession` y almacenó datos del usuario (`username`, `role`, `loginTime`).

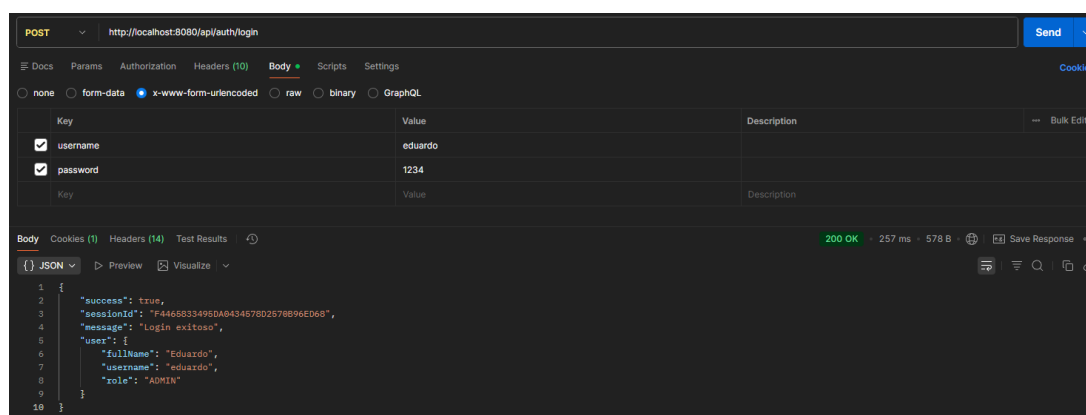


Figura 9: CP-04: Login y creación de sesión

12.2.5. CP-05: Información de sesión activa

Request:

GET http://localhost:8080/api/auth/session-info

Cookie: JSESSIONID=F4465833495DA0434578D2570B96ED68

Response:

```

1 {
2   "lastAccessedTime": "2026-01-25T02:38:31.105Z",
3   "authenticated": true,
4   "role": "ADMIN",
5   "loginTime": "2026-01-24T21:32:18.0013662",
6   "maxInactiveInterval": "1800 segundos",
7   "fullName": "Eduardo",
8   "sessionId": "F4465833495DA0434578D2570B96ED68",
9   "username": "eduardo"
10 }

```

Análisis: Los datos almacenados en la sesión persistieron entre requests. El servidor recuperó información mediante `session.getAttribute()`.



Figura 10: CP-05: Información de sesión activa

12.2.6. CP-06: Control de acceso sin sesión

Request (sin cookie):

POST http://localhost:8080/api/sales

Response:

```
1 HTTP/1.1 400 Bad Request
2
3 {
4   "message": "Required request body is missing"
5 }
```

Análisis: La solicitud fue rechazada antes de la validación de sesión, debido a la ausencia del cuerpo requerido en la petición. Spring no ejecutó el método del controlador, por lo que no se evaluó el estado de autenticación del usuario.

12.2.7. CP-07: Crear venta con JPA (CRUD completo)

Request:

```
1 POST http://localhost:8080/api/sales
2 Cookie: JSESSIONID=796E007A8F04B37D4ADA5196E3CE5334
3
4 {
5   "clientId": 1,
6   "details": [
7     {
8       "productId": 1,
9       "quantity": 2,
10      "discount": 10.0
11    }
12  ]
13 }
```

Listing 33: Body para crear venta

Response:

```
1 HTTP/1.1 201 Created
2
3 {
4   "data": {
5     "clientId": 1,
6     "details": [
7       {
8         "discount": 10.0,
9         "id": 1,
10        "price": 1299.99,
11        "productId": null,
12        "quantity": 2
13      }
14    ],
15    "number": 1,
16    "saleDate": "2026-01-24T22:13:21.3495754",
17    "total": 2589.98
18  },
19  "createdBy": "eduardo",
20  "success": true,
```

```

21     "message": "Venta creada exitosamente",
22     "saleNumber": 1,
23     "timestamp": "2026-01-24T22:13:21.6235878"
24 }

```

SQL generado por Hibernate:

```

1 INSERT INTO sale (client_id, sale_date, total) VALUES (?, ?, ?)
2 INSERT INTO sale_detail (sale_id, product_id, quantity,
3                          price, discount) VALUES (?, ?, ?, ?, ?)
4 UPDATE product SET stock = stock - ? WHERE codigo = ?

```

Listing 34: Queries ejecutadas por JPA

Análisis: La venta se creó mediante JPA validando stock, creando venta y detalles en cascada, y actualizando inventario en una transacción.

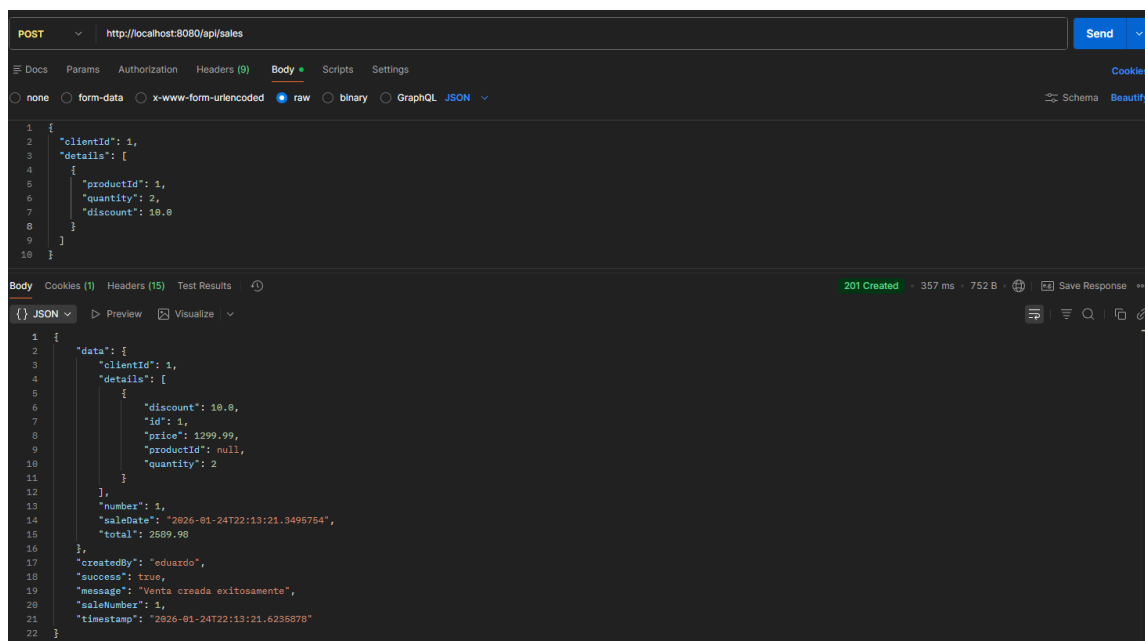


Figura 11: CP-07: Creación de venta con validación de sesión y rol

12.2.8. CP-08: Logout y destrucción de sesión

Request:

POST `http://localhost:8080/api/auth/logout`
 Cookie: `JSESSIONID=F4465833495DA0434578D2570B96ED68`

Response:

```

1 {
2     "success": true,
3     "message": "Sesi n cerrada exitosamente",
4     "user": "eduardo",
5     "timestamp": "2026-01-24T21:58:30.431606"
6 }

```

Análisis: El método `session.invalidate()` destruyó la sesión.

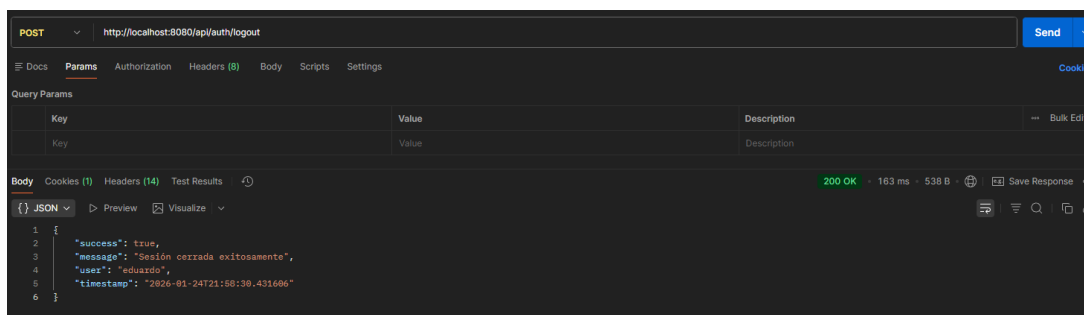


Figura 12: CP-08: Logout de sesi3n

12.3. Evidencias del sistema

Las pruebas ejecutadas validaron exitosamente:

- **Objetos Request/Response:** Captura de par3metros, headers y construcci3n de respuestas con c3digos HTTP apropiados
- **Gesti3n de sesiones:** Creaci3n, persistencia de datos, validaci3n de permisos y destrucci3n
- **Operaciones CRUD:** Inserci3n y consulta, mediante JPA
- **Procedimientos almacenados:** Ejecuci3n de l3gica compleja con validaciones y transacciones at3micas
- **Control de acceso:** Validaci3n de roles antes de ejecutar operaciones protegidas

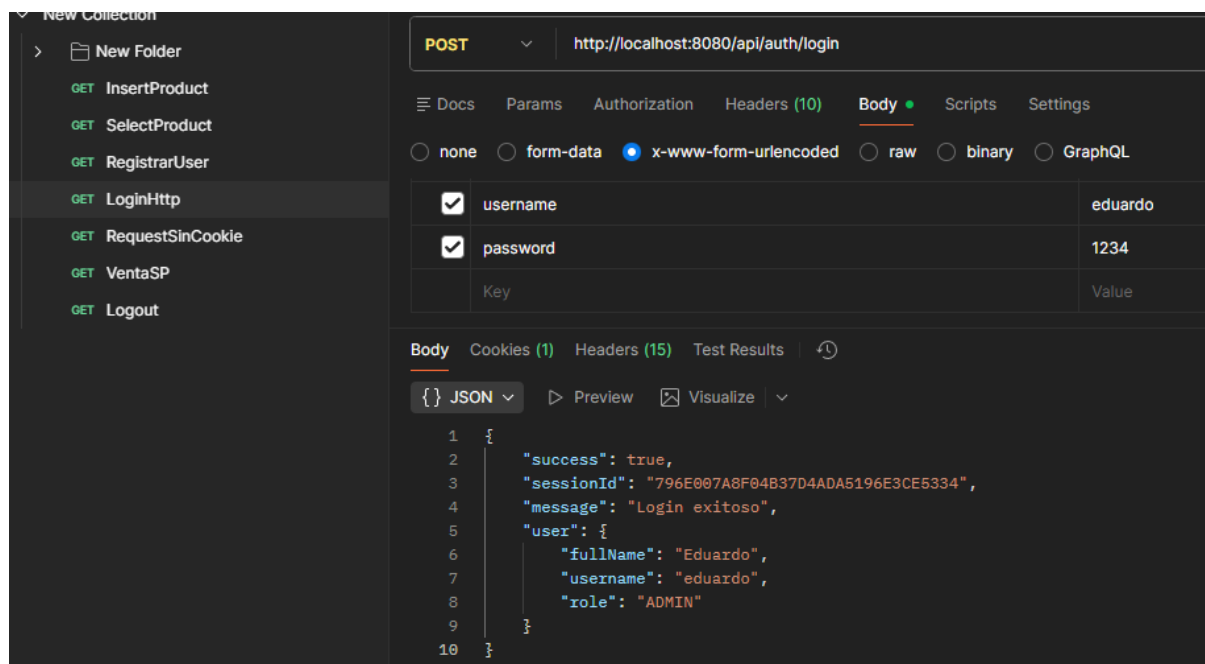


Figura 13: Colecci3n completa de Postman con todos los casos de prueba organizados

13. Conclusiones

La implementación de esta práctica experimental permitió consolidar conocimientos sobre la gestión de información en aplicaciones web mediante el uso de los objetos del ambiente (Request, Response, Session y Application) y técnicas avanzadas de acceso a datos, empleando Spring Boot como framework principal y PostgreSQL como sistema gestor de base de datos.

El manejo de Request y Response resultó fundamental para la construcción de APIs RESTful claras y robustas, permitiendo capturar parámetros, deserializar cuerpos JSON y personalizar respuestas HTTP mediante `ResponseEntity`. Asimismo, la gestión de sesiones evidenció su importancia en entornos multiusuario, al permitir mantener estado, controlar el acceso basado en roles y gestionar de forma segura el ciclo de vida de la sesión, aspectos clave para la seguridad de la aplicación.

En el acceso a datos, el uso de Spring Data JPA demostró la eficiencia de los frameworks ORM modernos, reduciendo código repetitivo y facilitando la implementación de operaciones CRUD mediante repositorios que extienden `JpaRepository`. La separación en capas (Controller → Service → Repository) mejoró la mantenibilidad del sistema y permitió centralizar la lógica de negocio y las validaciones.

El uso de procedimientos almacenados complementó a JPA al permitir ejecutar lógica compleja directamente en la base de datos. Procedimientos como `sp_process_sale`, `sp_get_sales_report` y `sp_update_product_stock` demostraron ventajas en términos de rendimiento, consistencia transaccional y reducción del tráfico de red, siendo especialmente útiles en reportes y operaciones concurrentes.

La arquitectura basada en el patrón MVC y la aplicación de buenas prácticas como el uso de `@Transactional`, la validación de datos y el manejo adecuado de excepciones contribuyeron a la construcción de un sistema escalable, mantenible y seguro. La integración de Spring Security permitió proteger endpoints sensibles y reforzó la importancia del control de acceso.

Finalmente, la comparación entre la implementación de lógica de negocio mediante JPA y procedimientos almacenados permitió identificar que ambos enfoques son válidos según el contexto. Mientras JPA ofrece mayor flexibilidad y facilidad de mantenimiento, los procedimientos almacenados destacan en escenarios donde el rendimiento y la integridad transaccional son críticos. En conjunto, esta práctica proporcionó una visión integral de los componentes y técnicas fundamentales para el desarrollo de aplicaciones web modernas y robustas.

Repositorio del proyecto: <https://github.com/ereinosov/practicaAppWeb>

Referencias

- [1] M. A. M. E. Abuhaliqa y C. Catal, “RESTful API Testing Methodologies: Rationale, Challenges, and Solution Directions,” *Applied Sciences*, vol. 12, n.º 9, pág. 4369, 2022, Open Access. DOI: 10.3390/app12094369.
- [2] R. Khan y K. McLaughlin, “A Secure Digital Image Marketplace: Microservices and OWASP API Security Using Spring Boot,” en *2024 International Conference on Computing and Information Technology (ICCIT)*, Open Access - IEEE Xplore, IEEE, 2024, págs. 1-6. DOI: 10.1109/ICISS62896.2024.10750956. dirección: <https://ieeexplore.ieee.org/document/10750956>.
- [3] M. A. Azad, M. S. Rahman y M. S. Islam, “Web Security and Web Application Security: Attacks and Prevention,” en *2023 International Conference on Electrical, Computer and Communication Engineering (ECCE)*, Open Access - IEEE Xplore, IEEE, 2023, págs. 1-6. DOI: 10.1109/ICACCS57279.2023.10112741. dirección: <https://ieeexplore.ieee.org/document/10112741>.
- [4] P. Perera, R. Silva e I. Perera, “A Full Stack Microservices Framework with Business Modelling,” en *2019 Moratuwa Engineering Research Conference (MERCon)*, Open Access - IEEE Xplore, IEEE, 2019, págs. 94-99. DOI: 10.1109/ICTER.2018.8615473. dirección: <https://ieeexplore.ieee.org/document/8615473>.
- [5] X. Liu, Y. Zhang y H. Wang, “Adaptive Load Balancing and Fault-tolerant Microservices Architecture for High-availability Web Systems Using Docker and Spring Cloud,” *Discover Applied Sciences*, vol. 7, n.º 1, págs. 1-18, 2025. DOI: 10.1007/s42452-025-07320-7.
- [6] “Spring Boot and Microservices in Cloud-Native Contexts,” *IRE Journals*, vol. 9, n.º 3, sep. de 2025, Open Access. dirección: <https://www.irejournals.com/formatedpaper/1710383.pdf>.
- [7] “Efficient Backend Development with Spring Boot,” *International Research Journal of Modernization in Engineering, Technology and Science*, vol. 9, n.º 3, mar. de 2025, Open Access. dirección: https://www.irjmets.com/uploadedfiles/paper/issue_3_march_2025/71070/final/fin_irjmets1743571164.pdf.
- [8] S. Karlsson, A. Causevic y D. Sundmark, “Exploring Behaviours of RESTful APIs in an Industrial Setting,” *Software Quality Journal*, jul. de 2024, Open Access - Springer. DOI: 10.1007/s11219-024-09686-0.
- [9] L. Zhang, Y. Wang y H. Chen, “Design and Implementation of Open Network Teaching System Based on Microservice Architecture,” en *2023 IEEE 6th International Conference on Computer and Communication Engineering Technology (CCET)*, Open Access - IEEE Xplore, IEEE, 2023, págs. 146-150. DOI: 10.1109/ICINC58035.2022.00030. dirección: <https://ieeexplore.ieee.org/document/10090016>.
- [10] Spring Framework Team, *Spring Data JPA Reference Documentation*, Official Documentation - Open Access, 2024. dirección: <https://docs.spring.io/spring-data/jpa/reference/index.html>.
- [11] Spring Framework Team, *Introduction to ORM with Spring Framework*, Official Documentation - Open Access, 2024. dirección: <https://docs.spring.io/spring-framework/reference/data-access/orm/introduction.html>.

-
- [12] S. Sakshi y A. Sharma, “Relational Database Performance Optimization Techniques,” en *Proceedings of the International Conference on Recent Advancements and Modernisations in Sustainable Intelligent Technologies and Applications (RAMSITA 2025)*, Open Access - Atlantis Press, Atlantis Press, mayo de 2025, págs. 112-124. DOI: 10.2991/978-94-6463-716-8_10. dirección: <https://www.atlantispress.com/proceedings/ramsita-25/126011462>.
 - [13] V. B. Ramu, “Optimizing Database Performance: Strategies for Efficient Query Execution and Resource Utilization,” *International Journal of Computer Trends and Technology*, vol. 71, n.º 7, págs. 15-21, 2023, Open Access - ResearchGate. DOI: 10.14445/22312803/IJCTT-V71I7P103. dirección: <https://www.researchgate.net/publication/372683874>.
 - [14] OWASP Foundation, *Session Management Cheat Sheet*, Open Access - OWASP Cheat Sheet Series, 2025. dirección: https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html.
 - [15] OWASP Foundation, *OWASP API Security Project*, Open Access - OWASP Foundation, 2025. dirección: <https://owasp.org/www-project-api-security/>.
 - [16] OWASP Foundation, *Testing for Concurrent Sessions*, Open Access - OWASP Web Security Testing Guide, 2025. dirección: https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/06-Session_Management_Testing/11-Testing_for_Concurrent_Sessions.
 - [17] S. Calzavara, R. Focardi, M. Nemec, A. Rabitti y M. Squarcina, “Measuring Web Session Security at Scale,” *Computers & Security*, vol. 110, págs. 102960, 2021, Open Access. DOI: 10.1016/j.cose.2021.102472.
 - [18] S. U. Meshram, “Evolution of Modern Web Services – REST API with its Architecture and Design,” *International Journal of Research in Engineering, Science and Management (IJRESM)*, vol. 4, n.º 7, págs. 83-86, jul. de 2021, Open Access. dirección: <https://journal.ijresm.com/index.php/ijresm/article/view/970>.
 - [19] H. Gurbuz y B. Tekinerdogan, “Automated Broken Object-Level Authorization Attack Detection in REST APIs Through OpenAPI to Colored Petri Nets Transformation,” *International Journal of Information Security*, feb. de 2025, Open Access - Springer. DOI: 10.1007/s10207-024-00970-5. dirección: <https://link.springer.com/article/10.1007/s10207-024-00970-5>.
 - [20] A. Bombarda, S. Bonfanti y A. Gargantini, “Integrating Formal Specifications in the Development and Testing of UIs by Formal Model-View-Controller Pattern,” *International Journal on Software Tools for Technology Transfer*, mayo de 2025, Open Access - Springer. DOI: 10.1007/s10009-025-00812-2. dirección: <https://link.springer.com/article/10.1007/s10009-025-00812-2>.
 - [21] I. Hussain, A. Abdulah y M. Rashid, “Evaluation of Web Application Session Security,” en *2020 International Conference on Innovation and Intelligence for Informatics, Computing and Technologies (3ICT)*, Open Access - IEEE Xplore, IEEE, 2020, págs. 1-6. DOI: 10.1049/cp.2019.0178. dirección: <https://ieeexplore.ieee.org/document/9068105>.

-
- [22] S. M. Alqhtani y M. N. Alenezi, "Automatic Detection and Risk Assessment of Session Management Vulnerabilities in Web Applications," en *2022 IEEE 12th Annual Computing and Communication Workshop and Conference (CCWC)*, Open Access - IEEE Xplore, IEEE, 2022, págs. 0846-0852. DOI: 10.1109/ICCKE54056.2021.9721455. dirección: <https://ieeexplore.ieee.org/document/9721455>.
 - [23] A. Bucko, M. S. E. Mahmoud y J. J. P. C. Rodrigues, "Enhancing JWT Authentication and Authorization in Web Applications Based on User Behavior History," *Computers*, vol. 12, n.º 4, pág. 78, 2023, Open Access - MDPI. DOI: 10.3390/computers12040078. dirección: <https://www.mdpi.com/2073-431X/12/4/78>.
 - [24] A. Sharma, R. Patel y S. Kumar, "Comparison of Different Authentication Techniques and Steps to Implement Robust JWT Authentication," en *2022 International Conference on Innovative Computing, Intelligent Communication and Smart Electrical Systems (ICSES)*, Open Access - IEEE Xplore, IEEE, 2022, págs. 1-6. DOI: 10.1109/ICCES54183.2022.9835796. dirección: <https://ieeexplore.ieee.org/document/9835796>.
 - [25] S. Tiwari y A. Jain, "An Authentication Based Scheme for Applications Using JSON Web Token," en *2020 International Conference on Communication and Signal Processing (ICCSP)*, Open Access - IEEE Xplore, IEEE, 2020, págs. 0622-0625. DOI: 10.1109/INMIC48123.2019.9022766. dirección: <https://ieeexplore.ieee.org/document/9022766>.
 - [26] M. Rana, A. Pandey, A. Mishra y V. Kandu, "Enhancing Data Security: A Comprehensive Study on the Efficacy of JSON Web Token (JWT) and HMAC SHA-256 Algorithm for Web Application Security," *International Journal on Recent and Innovation Trends in Computing and Communication*, vol. 11, n.º 11, págs. 170-178, 2023, Open Access. DOI: 10.17762/ijritcc.v11i9.9930. dirección: <https://ijritcc.org/index.php/ijritcc/article/view/9930>.
 - [27] A. Rahman, S. Islam y M. Hossain, "Performance and Security Comparison of Json Web Tokens (JWT) and Platform Agnostic Security Tokens (PASETO) on RESTful APIs," en *2023 IEEE International Conference on Cloud Computing and Services Science (CLOSER)*, Open Access - IEEE Xplore, IEEE, 2023, págs. 1-8. DOI: 10.1109/ICoCICs58778.2023.10277377. dirección: <https://ieeexplore.ieee.org/document/10277377>.
 - [28] M. C. Okoronkwo, O. S. Adewale y M. O. Adebisi, "A Systematic Review of Concurrency Control and Recovery Mechanisms in Distributed Database Systems: Trends, Techniques, and Performance Modelling," *International Journal of Advanced Computer Science and Applications*, vol. 16, n.º 1, págs. 45-62, 2025, Open Access. DOI: 10.52417/ojps.v6i2.1050. dirección: <https://thesai.org/Publications/ViewPaper?Volume=16&Issue=1&Code=IJACSA&SerialNo=7>.